Statistics With R Programming Assignment

Statistics with R Programming

ASSIGNMENT 1

.

Code of Conduct:

*I **RAKSHITH R.R (192324156)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



CO1

CO1: Design and construct appropriate R-based data representations using variables, data types, vectors, advanced data structures, matrix/array operations and R class mechanisms for data-oriented problems

---

Assignment 1 Title : Design and Develop a Scalable R Data Representation Framework for a University Academic Analytics System

---

Scenario — University Academic Analytics and Student Performance Platform

A university is developing an R-based Academic Analytics Platform to integrate student registration details, attendance, internal assessment marks, end-semester marks, grades, programme information, course categories and multi-dimensional assessment records. The system must support approximately 5,000 students and 25 subjects initially, but the architecture is expected to remain usable when the student population grows to 100,000.

The development team has not fixed the R data representation. Student information may be represented using atomic vectors, character vectors, logical vectors, lists, factors, matrices, arrays, data frames and user-defined R classes. The team is concerned that an inappropriate representation may cause unintended coercion, inconsistent data types, difficult updates, weak validation, inefficient operations or poor maintainability.

You have been appointed as the R Data Architecture Team. Your responsibility is not merely to demonstrate R syntax. You must investigate the requirements, design a complete representation framework, construct reusable R components, integrate different R objects, validate the design experimentally and defend the final architecture using evidence.

The final solution must clearly distinguish between what R permits, what R automatically converts or coerces, and what is appropriate for the university application. Students are expected to make design decisions independently and justify why their constructed solution is suitable for the given scenario.

Design Challenge

Create and validate an integrated R data-representation framework for the university system. The framework must define how each category of information is stored, how multi-dimensional academic data is represented, how objects are validated and manipulated, and how S3, S4 or Reference Class mechanisms can be incorporated where a structured object model is required.

Your final design must be executable in R and must demonstrate interaction among the selected data structures rather than treating each R object as an isolated example. Every major design decision must be supported by experimental evidence and a clear explanation of the trade-offs involved.

TASK 1 – Formulate the System Enquiry and Create the Data Architecture Blueprint 4 Marks

- Convert the scenario into a formal set of data-representation requirements. Identify the information categories, relationships among them, expected operations and constraints.
- Create a data-architecture blueprint showing how variables, atomic/character/logical vectors, lists, factors, matrices, arrays and data frames will cooperate in the proposed system.
- Formulate at least four enquiry questions that your design must answer experimentally, such as how representation affects coercion, indexing, validation, updating and analytical operations.
- Define at least five design criteria and use them as acceptance criteria for the final framework.

Evidence Expected

A requirements-to-representation blueprint, enquiry questions, design criteria and a clear rationale showing how the proposed architecture addresses the scenario.

TASK 2 – Design a Robust Student Core-Data Representation **4 Marks**

- Design and construct the R representation for registration number, student name, attendance, eligibility/pass-fail status, programme and other essential attributes.
- Create suitable atomic, character and logical vectors and establish naming and type-consistency rules.
- Construct validation logic that detects unsuitable values, missing values and unintended type conversion.
- Demonstrate at least two alternative representations for one important attribute, then create the final representation based on the identified requirements.

Evidence Expected

Executable R objects, validation rules, test cases, rejected alternatives and a justified final student-data representation.

TASK 3 – Create a Coercion-Safe Data Preparation Module **4 Marks**

- Create an R module that deliberately tests mixed numeric, integer, character and logical data and records the resulting types.
- Design rules or helper functions to prevent or detect harmful automatic coercion before the data enter the academic analytics system.
- Construct test cases containing valid, invalid, missing and mixed-type values and demonstrate how the module responds.
- Explain how your design preserves semantic meaning while still using R's native data structures.

Evidence Expected

A reusable coercion-testing/validation component, experimental outputs, test cases, and evidence that the constructed rules prevent or expose unintended conversion.

TASK 4 – Construct a Student-by-Subject Matrix Processing Engine **4 Marks**

- Create a matrix-based component representing student marks across subjects.
- Develop R operations for indexing, row/column summaries, transposition and at least one meaningful matrix operation relevant to academic analytics.
- Construct a mechanism for connecting matrix positions with student and subject identifiers without losing interpretability.
- Create and justify a conversion strategy for moving between the matrix representation and a more metadata-rich representation such as a data frame.

Evidence Expected

Working matrix component, meaningful operations, mapping/identifier strategy, conversion code and evidence demonstrating correct results.

TASK 5 – Create a Multi-Dimensional Assessment Array Model **4 Marks**

- Design and construct a three-dimensional or higher-dimensional array for students, subjects and assessment components such as internal, model and practical marks.
- Develop indexing and calculation operations that retrieve and summarize selected dimensions.
- Create a method for assigning meaningful dimension names and demonstrate why the dimensions remain interpretable.
- Construct at least one transformation from the array into another R representation suitable for reporting or analysis.

Evidence Expected

Executable array model, named dimensions, retrieval/calculation operations, transformation procedure and evidence of correct multi-dimensional handling.

TASK 6 – Build an Integrated List, Factor and Data-Frame Layer **5 Marks**

- Create a heterogeneous student record using a list and design factors for categorical variables such as programme, grade or course category.
- Construct a data frame that integrates appropriate vectors, factors and numerical variables into an analysis-ready table.
- Develop operations for adding, updating, selecting and summarising student records.
- Create rules for deciding when a variable should remain a character vector and when it should be represented as a factor.
- Demonstrate that the constructed layer can exchange data with the matrix and array components developed earlier.

Evidence Expected

An integrated R data layer, reusable operations, categorical-data design rules, and evidence showing successful interaction among lists, factors, data frames, matrices and arrays.

TASK 7 – **Design an Object-Oriented Student/Course Model Using R Classes** **5 Marks**

- Create a realistic Student, Course or AcademicRecord object model using S3, S4 or Reference Class mechanisms.
- Implement the selected class structure with appropriate attributes and behaviour for the scenario.
- Create at least two operations/methods or functions that act on the constructed object, such as updating marks, calculating eligibility, generating a summary or validating an academic record.

• Construct a small comparative prototype using a second class mechanism and use the investigation to justify the final class design.
• Explain how the chosen class mechanism improves structure, validation, reuse or maintainability for the proposed system.

Evidence Expected
Constructed class-based prototype(s), working methods/functions, test cases, comparison evidence and a technically justified final class architecture.

TASK 8 – Develop a Reusable R Function Library for the Framework 4 Marks

• Create a minimum of four reusable R functions supporting the proposed framework, including at least one function for validation and one for academic calculation or summary.
• Use default arguments where appropriate and explicitly design the return object for each function.
• At least one function must return a complex object such as a list or data frame and must integrate with another component of the framework.
• Create test cases that demonstrate valid, boundary and invalid inputs and document the expected behaviour.

Evidence Expected
A reusable function library, documented arguments/return values, test cases and evidence of integration with the constructed data framework.

TASK 9 – Integrate, Test and Refine the Complete R Framework 4 Marks

• Integrate the components created in Tasks 2–8 into one coherent academic analytics prototype.
• Generate or construct a representative dataset and execute a realistic workflow from raw student information to validated, structured and analysis-ready objects.
• Design systematic test cases covering correctness, missing values, invalid categories, dimension errors and type/coercion problems.
• Record failures encountered during development and show how the design was refined to address them.

Evidence Expected
A reproducible integrated prototype, workflow demonstration, systematic test cases, failure/refinement evidence and final corrected implementation

TASK 10 – Create an Evidence-Based Scalability and Maintainability Evaluation 4 Marks

- Construct datasets representing at least 5,000 and 100,000 students, with an appropriate number of subjects and assessment dimensions.
- Experiment with at least three important operations from the proposed framework and record meaningful measures such as execution time, object dimensions, memory-related observations where feasible, or number of processing steps.
- Create a comparison table and at least one suitable graph or structured visualization.
- Use the evidence to refine the architecture where necessary rather than simply reporting performance results.

Evidence Expected

Reproducible experimental setup, comparison table, visualization, interpretation and evidence of at least one design refinement based on the experiments.

TASK 11 – Defend the Final Architecture Under a Changed Requirement 4 Marks

- Changed Requirement: The university now plans to integrate 100,000 students, 40 subjects, continuous assessment components and frequent individual-record updates. The analytics team also requires the framework to remain easy to validate, transform and extend.
- Create a revised version of your architecture by deciding which components should be retained, modified, replaced or redesigned.
- Construct at least one new or modified component that directly addresses the changed requirement and demonstrate its integration with the existing framework.
- Defend the final architecture using evidence from your experiments, object behaviour, scalability observations, validation requirements, maintainability and semantic correctness.
- Identify at least two trade-offs or limitations that remain in your final design and propose future improvements.

Evidence Expected

A revised executable architecture, demonstrated adaptation, evidence-based defence, explicit trade-offs and a clear explanation of how the final design satisfies the changed requirements.

MARK DISTRIBUTION

Task	Marks
TASK 1 – Formulate	4
TASK 2 – Design	4
TASK 3 – Create	4
TASK 4 – Construct	4
TASK 5 – Create	4
TASK 6 – Build	5
TASK 7 – Design	5
TASK 8 – Develop	4
TASK 9 – Integrate,	4

TASK 10 – Create	4
TASK 11 – Defend	4
TOTAL	50

SOLUTION REPORT - ASSIGNMENT 1

Design and Develop a Scalable R Data Representation Framework for a University Academic Analytics System

This report follows the assignment sheet above. Each task has its own section containing the approach, the executable R code, the output captured by running that code, figures where they help, and a justification based on that evidence.

Execution environment and conventions

- All code was run in R 4.3.3 using base R only (packages stats, methods, graphics); no add-on package is needed.
- Tasks 1 to 11 form one framework. They were run in order in a single R session, so later tasks reuse objects created earlier (core, marks, arr, total, analysis, the Task 3 and Task 8 functions and the S4 class). Run the code blocks in the same order to reproduce the results.
- Random data uses fixed seeds. Timings in Tasks 7, 10 and 11 depend on the machine, so another computer will show different seconds but the same pattern.
- Mark scheme used in the demonstration: pass mark 40, attendance minimum 75%, grade bands F (below 40), C, B, A, O (85 and above), component weights internal 30% / model 30% / practical 40%.

Task map

Task	Component built	Marks	Main evidence
1	Requirements, blueprint, enquiry questions, design criteria	4	Blueprint figure, 4 experiments, criteria table
2	Student core-data vectors and validation	4	Schema, 3 attendance alternatives, 11 detected issues
3	Coercion-safe data preparation module	4	Type-risk table, 6 conversion test cases
4	Student-by-subject matrix engine	4	Operations, ID mapping, heat map, round trip
5	Students x subjects x components array	4	Named dimensions, 4 transformations, box plot
6	List, factor and data-frame layer	5	Record, factors, CRUD operations, exchange tests
7	S4 Student class and S3 comparison	5	5 rejected invalid objects, comparison table
8	Reusable function library	4	5 functions, 19 of 19 tests passed
9	Integrated prototype and refinement	4	10 of 10 tests, 8 failures recorded and fixed
10	Scalability at 5,000 and 100,000 students	4	Timing table, graph, vectorised refinement

11	Changed requirement: 100,000 x 40 with updates	4	Reference Class store, acceptance re-test

TASK 1 - Formulate the System Enquiry and Create the Data Architecture Blueprint (4 Marks)

1.1 Requirements derived from the scenario

The scenario is converted into formal data-representation requirements. Each requirement says what must be stored and what that forces on the R representation.

ID	Information category	Requirement	Representation consequence
R1	Identity: registration number, name, programme	Unique labels, never used in arithmetic; programme from a fixed list	Character vectors; programme later a factor with 4 levels
R2	Attendance and eligibility	Percentage 0-100; eligibility derived from attendance (at least 75)	Numeric vector; logical vector (unknown stays NA)
R3	Marks per student per subject	Rectangular, numeric, fast row and column summaries	Numeric matrix with row and column names
R4	Marks per student, subject and assessment component	Select by any dimension by name	3-D array with named dimensions
R5	Programme, grade, course category	Few repeated labels; grades have an order	Factor and ordered factor
R6	One heterogeneous student record	Mixed types in one object; validated structure	List; S4 object when validity is required
R7	Growth from 5,000 to 100,000 students; frequent updates	Vectorised operations, predictable memory, cheap single-record update	Tested in Tasks 10 and 11
R8	Quality: unintended coercion and weak validation	Detect conversions and invalid values before analysis	Typed ingestion plus validator functions

Relationships. One student has many marks (student, subject, component). A programme groups many students. A subject belongs to a course category. Eligibility is derived from attendance and marks, so it is computed, not typed in.

Expected operations. Ingest, validate, index by name, update one record, summarise by row, column or group, convert between representations, and report.

Constraints. Base R only; identifiers stored as text; marks and attendance within 0-100; reproducible runs; the same design must still work at 100,000 students.

1.2 Data-architecture blueprint

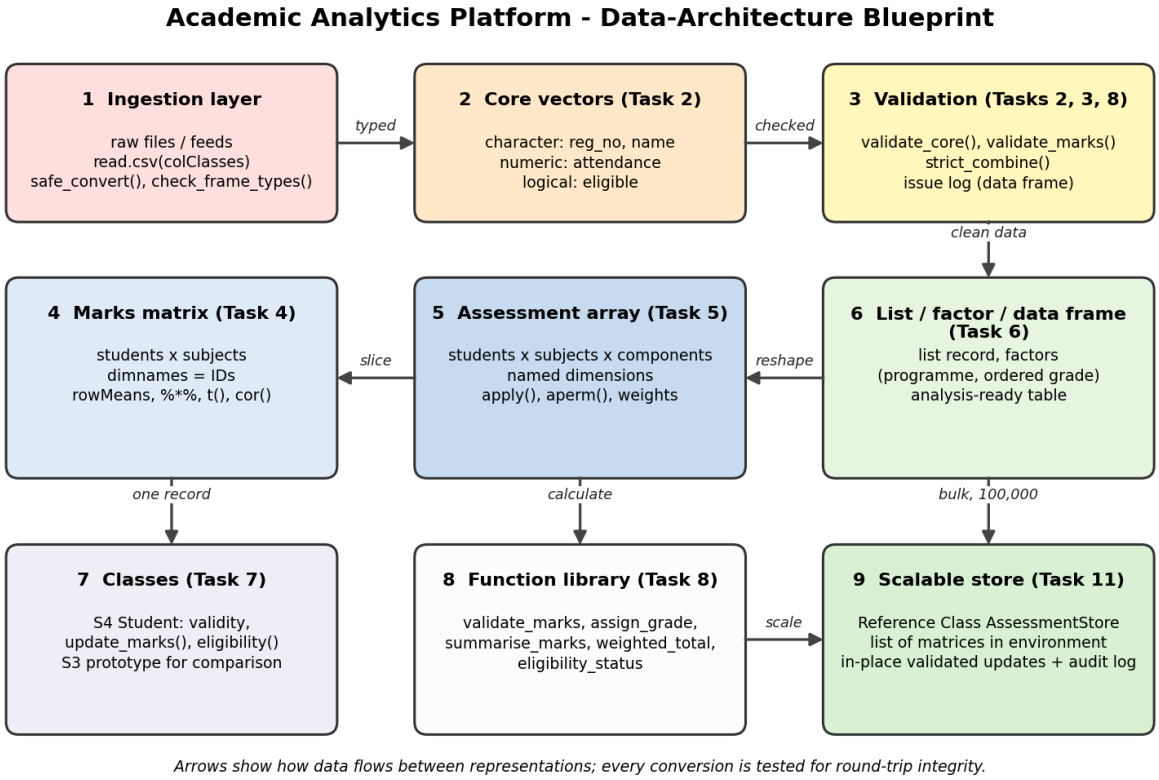


Figure 1: Blueprint showing how vectors, matrices, arrays, lists, factors, data frames and classes cooperate

Raw data enters through the ingestion layer, passes typed core vectors and validation, and then feeds three structural views of the same marks: a matrix (one component), an array (all components) and a data frame (metadata-rich). Classes and the function library act on single records and on the structures. The scalable store of Task 11 replaces the array for bulk updating.

1.3 Enquiry questions answered experimentally

- **EQ1** How does the choice of representation affect coercion?
- **EQ2** How does the representation affect indexing?
- **EQ3** How does the representation affect validation of values?
- **EQ4** How does the representation affect updating and categorical data?

R code

```
# TASK 1: Enquiry experiments behind the architecture blueprint
options(width = 85)
```

```
cat("=== EQ1: How does representation affect coercion? ===\n")
mixed <- list(num_chr = c(85.5, "A"), num_lgl = c(85.5, TRUE),
              int_num = c(5L, 2.5),   lgl_chr = c(TRUE, "Pass"),
              int_lgl = c(3L, FALSE))
coercion_tab <- data.frame(
  expression   = names(mixed),
  result_class = sapply(mixed, class),
  result_value = sapply(mixed, function(v) paste(v, collapse = " | ")),
  row.names = NULL)
print(coercion_tab)
rec <- list(reg_no = "192324001", attendance = 85.5, eligible = TRUE)
cat("\nThe same values inside a list keep their own types:\n")
print(sapply(rec, class))

cat("\n=== EQ2: How does representation affect indexing? ===\n")
marks_vec <- c(S1 = 78, S2 = 65, S3 = 90)
marks_mat <- matrix(c(78, 65, 90, 55, 70, 82), nrow = 3,
                    dimnames = list(c("S1", "S2", "S3"), c("MA", "CS")))
cat("Vector by name   marks_vec['S2']       :", marks_vec["S2"], "\n")
cat("Matrix by name   marks_mat['S2','CS']  :", marks_mat["S2", "CS"], "\n")
cat("marks_mat[1, ] class                   :", class(marks_mat[1, ]), "\n")
cat("marks_mat[1, , drop=FALSE] class       :", class(marks_mat[1, , drop = FALSE]), "\n")

cat("\n=== EQ3: How does representation affect validation? ===\n")
att <- c(92, 88, NA, 105, -4, 76)
cat("mean(att)               :", mean(att), "\n")
cat("mean(att, na.rm = TRUE) :", mean(att, na.rm = TRUE),
    "(silently includes 105 and -4)\n")
ok <- !is.na(att) & att >= 0 & att <= 100
cat("validated mean          :", mean(att[ok]), "| rejected positions:",
    paste(which(!ok), collapse = ", "), "\n")

cat("\n=== EQ4: How does representation affect updating? ===\n")
prog <- factor(c("AI&DS", "CSE", "AI&DS"), levels = c("AI&DS", "CSE"))
res <- withCallingHandlers({ prog[2] <- "ECE"; prog },
  warning = function(w) { cat("Warning:", conditionMessage(w), "\n")
```

```
                    invokeRestart("muffleWarning") })
cat("Factor after assigning an unknown level:\n"); print(res)
f <- factor(c("10", "20", "30", "20"))
cat("as.numeric(factor)               :", as.numeric(f), "\n")
cat("as.numeric(as.character(factor)) :", as.numeric(as.character(f)), "\n")

cat("\n=== Design criteria used as acceptance criteria ===\n")
design_criteria <- data.frame(
  id = paste0("C", 1:6),
  criterion = c("Type safety", "Validity", "Interpretability",
                "Round-trip integrity", "Scalability", "Maintainability"),
  acceptance_test = c(
    "No silent coercion; all type defects detected",
    "All invalid / out-of-range records flagged",
    "All dimensions named; lookup by ID works",
    "matrix/data frame/array conversions lossless",
    "100,000 students: < 10 s per op, < 1 GB",
    "Update = 1 validated call; add component, no rewrite"))
print(design_criteria, right = FALSE)
```

Output (captured from running the code above)

```
=== EQ1: How does representation affect coercion? ===
  expression result_class result_value
1    num_chr    character     85.5 | A
2    num_lgl      numeric     85.5 | 1
3    int_num      numeric      5 | 2.5
4    lgl_chr    character  TRUE | Pass
5    int_lgl      integer        3 | 0


The same values inside a list keep their own types:
     reg_no  attendance    eligible
"character"   "numeric"   "logical"

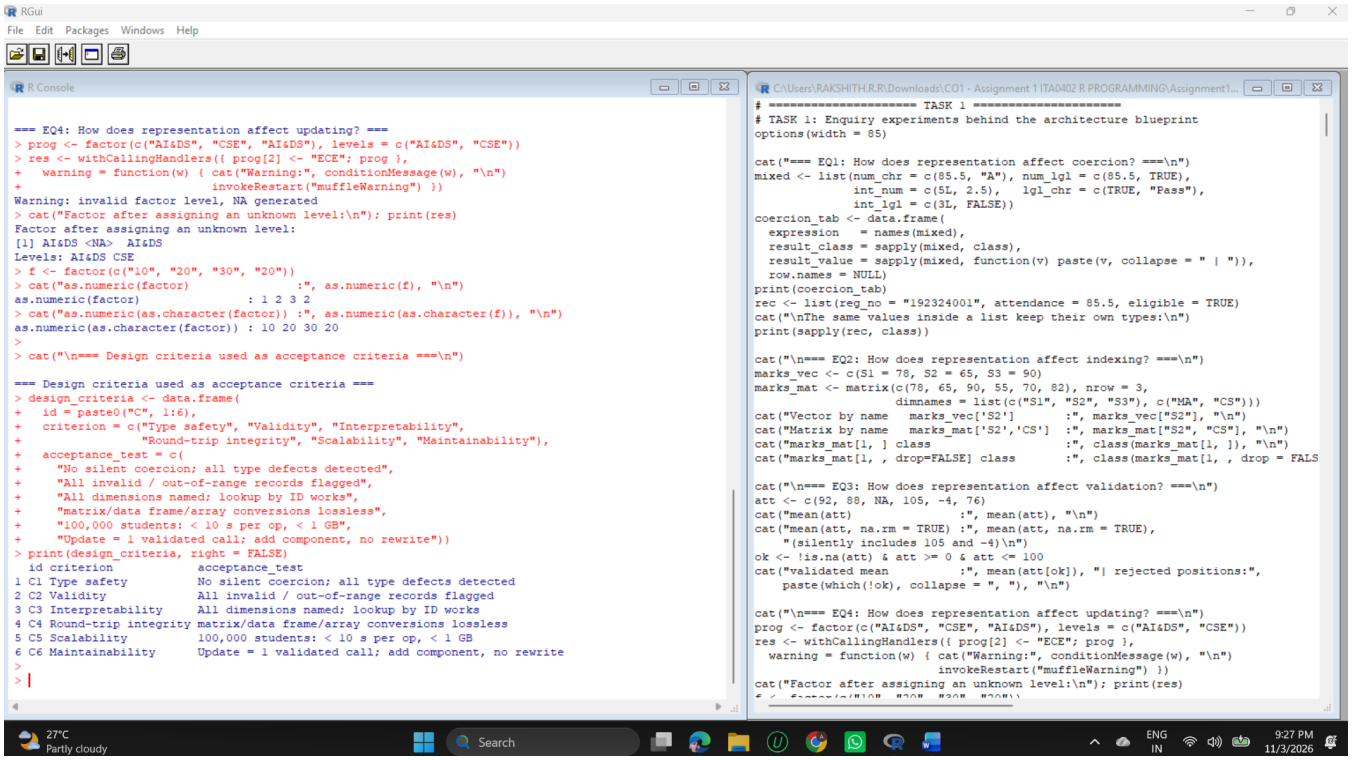

=== EQ2: How does representation affect indexing? ===
```

```
Vector by name   marks_vec['S2']       : 65
Matrix by name   marks_mat['S2','CS']  : 70
marks_mat[1, ] class                   : numeric
marks_mat[1, , drop=FALSE] class       : matrix array


=== EQ3: How does representation affect validation? ===
mean(att)               : NA
mean(att, na.rm = TRUE) : 71.4 (silently includes 105 and -4)
validated mean          : 85.33333 | rejected positions: 3, 4, 5


=== EQ4: How does representation affect updating? ===
Warning: invalid factor level, NA generated
Factor after assigning an unknown level:
[1] AI&DS <NA>  AI&DS
Levels: AI&DS CSE
as.numeric(factor)              : 1 2 3 2
as.numeric(as.character(factor)) : 10 20 30 20


=== Design criteria used as acceptance criteria ===
  id criterion           acceptance_test
1 C1 Type safety         No silent coercion; all type defects detected
2 C2 Validity            All invalid / out-of-range records flagged
3 C3 Interpretability    All dimensions named; lookup by ID works
4 C4 Round-trip integrity matrix/data frame/array conversions lossless
5 C5 Scalability         100,000 students: < 10 s per op, < 1 GB
6 C6 Maintainability     Update = 1 validated call; add component, no rewrite
```



1.4 Findings

- **EQ1:** c(85.5, "A") silently becomes character, and c(85.5, TRUE) turns TRUE into 1. A list keeps each element's own type, so heterogeneous records belong in lists, not in vectors.
- **EQ2:** Names work as indexes for vectors and matrices. marks_mat[1,] drops to a plain numeric vector while drop = FALSE keeps a matrix, so matrix code must say which one it wants.
- **EQ3:** mean(att, na.rm = TRUE) returned 71.4 and silently included 105 and -4. After validation the mean is 85.33 and positions 3, 4 and 5 are reported as rejected. Representation alone does not validate; explicit rules are needed.
- **EQ4:** Assigning an unknown level to a factor produced NA with a warning, and as.numeric() on a factor returned the level codes (1 2 3 2) instead of the values (10 20 30 20). Factors are suitable for fixed categories and dangerous for numbers.

1.5 Design criteria used as acceptance criteria

The six criteria in the output above are measurable. They are tested on the Task 9 prototype and re-tested in Task 11, where all six pass on the revised architecture.

ID	Criterion	Acceptance test	Re-tested in
C1	Type safety	No silent coercion; all type defects detected	Tasks 3, 9, 11
C2	Validity	All invalid or out-of-range records flagged	Tasks 2, 7, 8, 9, 11
C3	Interpretability	All dimensions named; lookup by ID works	Tasks 4, 5, 11

C4	Round-trip integrity	matrix, data frame and array conversions are lossless	Tasks 4, 5, 6, 9, 11
C5	Scalability	100,000 students: under 10 s per operation and under 1 GB	Tasks 10, 11
C6	Maintainability	Update is one validated call; a component is added without rewriting	Task 11

1.6 Rationale

Typed vectors protect identifiers and measurements, the matrix and array give fast, name-addressable numeric work, factors and data frames give analysis-ready categorical data, lists and S4 objects hold heterogeneous validated records, and explicit validators cover what R's types alone cannot check. Every structure is linked to the others by tested conversions, which is what makes it a framework and not a set of isolated examples.

TASK 2 - Design a Robust Student Core-Data Representation (4 Marks)

Concept. In R, a vector has one type. Choosing that type decides which operations are correct, which comparisons are meaningful and which errors appear silently. The core attributes are therefore given fixed types, naming rules and a validator.

- **Naming rules:** lower-case snake_case (reg_no, attendance); one vector per attribute; same length for all vectors.
- **Type rules:** reg_no, name, programme character; attendance numeric (percentage); eligible logical derived from attendance.
- **Identifier rule:** the registration number is text because it is a label. It is never summed, and text keeps leading zeros that integers would lose (Task 3 shows this happening on import).

R code

```
# TASK 2: Student core-data representation
options(width = 85)
valid_programmes <- c("AI&DS", "CSE", "IT", "ECE")

reg_no     <- sprintf("1923240%02d", 1:12)
name       <- c("Aarav Kumar", "Bhavya Rao", "Charan Teja", "Divya Nair",
               "Eshwar Pillai", "Farhan Ali", "Gayathri S", "Harini M",
               "Ishaan Verma", "Jaya Lakshmi", "Karthik R", "Lavanya P")
programme  <- c("AI&DS", "AI&DS", "CSE", "IT", "ECE", "AI&DS",
               "CSE", "IT", "AI&DS", "ECE", "CSE", "AI&DS")
attendance <- c(92, 88, 74.5, 96, 81, 69, 90, 75, 85.5, 78, 100, 83)
eligible   <- attendance >= 75

cat("--- Naming and type-consistency rules (schema) ---\n")
schema <- data.frame(
  attribute = c("reg_no", "name", "programme", "attendance", "eligible"),
  r_type    = c("character", "character", "character", "numeric", "logical"),
  rule      = c("9 digits, kept as text (IDs are labels, never arithmetic)",
                "non-empty text", "one of AI&DS/CSE/IT/ECE",
                "percentage 0-100", "TRUE if attendance >= 75"))
print(schema, right = FALSE)

cat("\n--- Alternative representations of ATTENDANCE ---\n")
```

```
att_chr <- c("92%", "100%", "75%", "68.5%", "88%")
att_num <- c(92, 100, 75, 68.5, 88)
attended <- c(46L, 50L, 38L, 34L, 44L); total <- 50L
cat("A) character  sort():", sort(att_chr), "\n")
cat("B) numeric    sort():", sort(att_num), "\n")
cat("A) '100%' >= '75%' ?", "100%" >= "75%", " (wrong: text comparison)\n")
cat("B) 100 >= 75 ?      ", 100 >= 75, "\n")
mA <- tryCatch(mean(att_chr),
               warning = function(w) paste("WARNING ->", conditionMessage(w)))
cat("A) mean(character)  :", mA, "\n")
cat("B) mean(numeric)    :", mean(att_num), "\n")
cat("C) attended/total*100 (two integer vectors):",
    attended / total * 100, "\n")
set.seed(1); big <- round(runif(1e5, 50, 100), 1)
cat("Memory for 100,000 values -> character:",
    format(object.size(as.character(big)), units = "Kb"),
    "| numeric:", format(object.size(big), units = "Kb"), "\n")

alt <- data.frame(
  option = c("A character '92%'", "B numeric 92", "C integer pair"),
  arith_ok = c("No", "Yes", "Yes (derived)"), compare_ok = c("No", "Yes", "Yes (derived)"),
  n_vectors = c(1, 1, 2), effort = c("High", "Low", "Medium"))
print(alt, right = FALSE)
cat("DECISION: B (numeric percentage). A fails arithmetic and comparison;",
    "C needs two synchronised vectors.\n")

validate_core <- function(reg_no, name, attendance, programme,
                          valid_programmes = c("AI&DS", "CSE", "IT", "ECE")) {
  issues <- data.frame(row = integer(), field = character(),
                       problem = character(), stringsAsFactors = FALSE)
  add <- function(rows, field, problem)
    if (length(rows) > 0)
      issues <<- rbind(issues, data.frame(row = rows, field = field,
                                          problem = problem))
  if (!is.character(reg_no))
    add(NA_integer_, "reg_no", paste("type is", class(reg_no)[1]))
```

```
  reg_chr <- as.character(reg_no)
  add(which(!grepl("^[0-9]{9}$", reg_chr)), "reg_no", "not a 9-digit ID")
  add(which(duplicated(reg_chr)), "reg_no", "duplicate ID")
  add(which(is.na(name) | trimws(name) == ""), "name", "missing or empty")
  att_num  <- suppressWarnings(as.numeric(attendance))
  not_num  <- which(is.na(att_num) & !is.na(attendance))
  add(not_num, "attendance", "non-numeric text (unintended conversion)")
  add(which(is.na(attendance)), "attendance", "missing value")
  add(which(!is.na(att_num) & (att_num < 0 | att_num > 100)),
      "attendance", "outside 0-100")
  add(which(is.na(programme) | !programme %in% valid_programmes),
      "programme", "invalid category")
  issues
}

cat("\n--- Test case 1: clean data ---\n")
iss <- validate_core(reg_no, name, attendance, programme)
cat("Issues found:", nrow(iss), "\n")

cat("\n--- Test case 2: dirty data ---\n")
bad_reg  <- c(reg_no[1:3], "19232", reg_no[2], reg_no[6:8])
bad_name <- c("A", "B", NA, "D", " ", "F", "G", "H")
bad_att  <- c("92", "85%", NA, "120", "-3", "abc", "75", "88")
bad_prog <- c("AI&DS", "CSE", "MECH", "IT", NA, "ECE", "CSE", "AI&DS")
print(validate_core(bad_reg, bad_name, bad_att, bad_prog))

core <- data.frame(reg_no = reg_no, name = name, programme = programme,
                   attendance = attendance, eligible = eligible,
                   stringsAsFactors = FALSE)
cat("\n--- Final student core data ---\n")
str(core)
print(head(core, 4))
```

Output (captured from running the code above)

```
--- Naming and type-consistency rules (schema) ---
```

```
  attribute  r_type    rule
1 reg_no     character 9 digits, kept as text (IDs are labels, never arithmetic)
2 name       character non-empty text
3 programme  character one of AI&DS/CSE/IT/ECE
4 attendance numeric   percentage 0-100
5 eligible   logical   TRUE if attendance >= 75

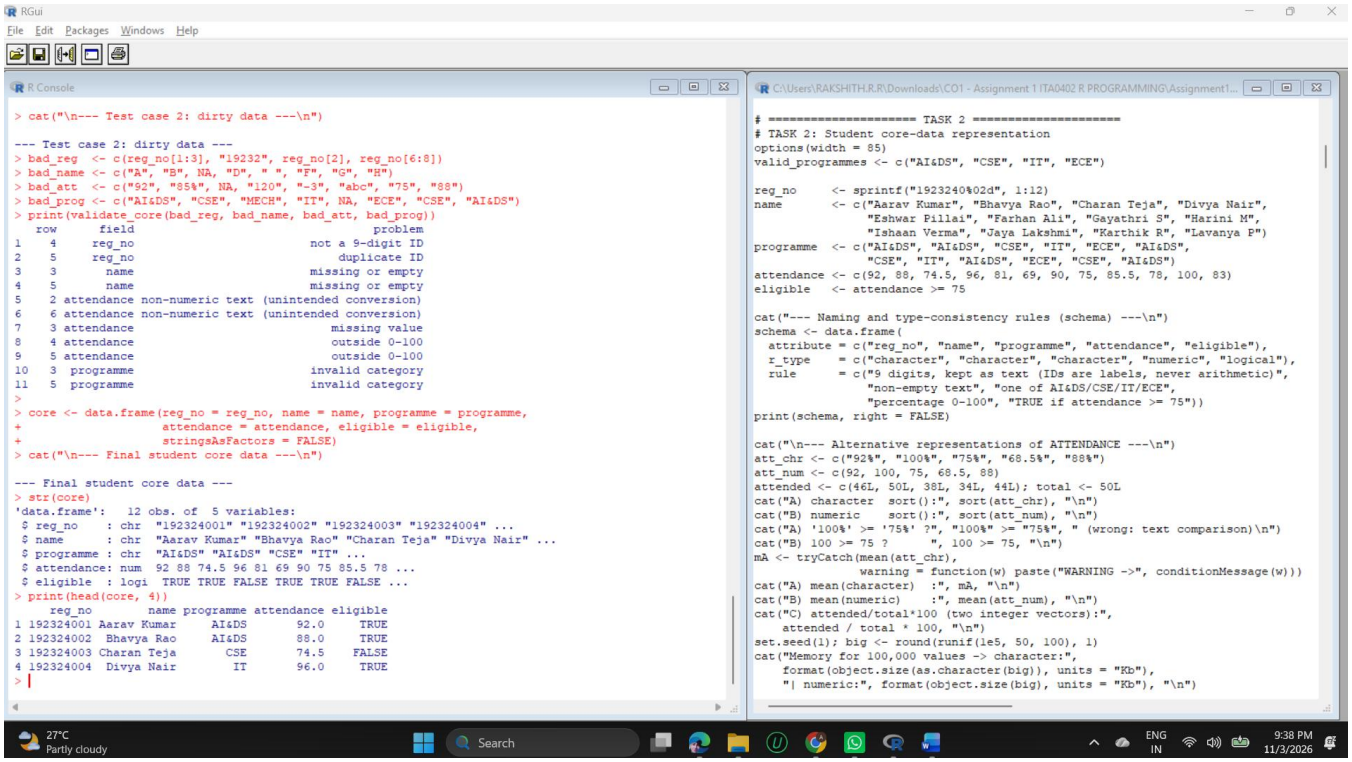
--- Alternative representations of ATTENDANCE ---
A) character  sort(): 100% 68.5% 75% 88% 92%
B) numeric    sort(): 68.5 75 88 92 100
A) '100%' >= '75%' ? FALSE  (wrong: text comparison)
B) 100 >= 75 ?       TRUE
A) mean(character)  : WARNING -> argument is not numeric or logical: returning NA
B) mean(numeric)    : 84.7
C) attended/total*100 (two integer vectors): 92 100 76 68 88
Memory for 100,000 values -> character: 808.7 Kb | numeric: 781.3 Kb
  option           arith_ok      compare_ok    n_vectors effort
1 A character '92%' No           No            1         High
2 B numeric 92      Yes          Yes           1         Low
3 C integer pair    Yes (derived) Yes (derived) 2         Medium
DECISION: B (numeric percentage). A fails arithmetic and comparison; C needs two synchronised vectors.

--- Test case 1: clean data ---
Issues found: 0

--- Test case 2: dirty data ---
   row      field                          problem
1    4     reg_no                   not a 9-digit ID
2    5     reg_no                      duplicate ID
3    3       name                   missing or empty
4    5       name                   missing or empty
5    2 attendance non-numeric text (unintended conversion)
6    6 attendance non-numeric text (unintended conversion)
7    3 attendance                      missing value
8    4 attendance                      outside 0-100
9    5 attendance                      outside 0-100
```

```
10   3  programme                 invalid category
11   5  programme                 invalid category


--- Final student core data ---
'data.frame':    12 obs. of  5 variables:
 $ reg_no    : chr  "192324001" "192324002" "192324003" "192324004" ...
 $ name      : chr  "Aarav Kumar" "Bhavya Rao" "Charan Teja" "Divya Nair" ...
 $ programme : chr  "AI&DS" "AI&DS" "CSE" "IT" ...
 $ attendance: num  92 88 74.5 96 81 69 90 75 85.5 78 ...
 $ eligible  : logi  TRUE TRUE FALSE TRUE TRUE FALSE ...
    reg_no       name programme attendance eligible
1 192324001 Aarav Kumar     AI&DS       92.0     TRUE
2 192324002  Bhavya Rao     AI&DS       88.0     TRUE
3 192324003 Charan Teja       CSE       74.5    FALSE
4 192324004  Divya Nair        IT       96.0     TRUE
```



Alternative representations of attendance

Option	Result in the experiment	Verdict

A character "92%"	sort() placed "100%" first; "100%" >= "75%" gave FALSE; mean() returned NA with a warning	Rejected: wrong comparisons and no arithmetic
B numeric 92	Sorted correctly, 100 >= 75 gave TRUE, mean of the sample was 84.7	**Selected**
C two integer vectors (attended, total)	Percentage derived correctly (92, 100, 76, 68, 88) but two vectors must stay synchronised	Rejected as the stored form; acceptable as a raw source

Memory was measured for 100,000 values (character 808.7 Kb, numeric 781.3 Kb). The difference is small, so memory did not decide the choice; correctness of comparison and arithmetic did.

Validation logic and test cases

- validate_core() returns a data frame of issues (row, field, problem) instead of stopping at the first error, so every defect is reported at once.
- **Test case 1 (clean data):** 0 issues.
- **Test case 2 (dirty data):** 11 issues found across four fields: a short ID and a duplicate ID, two missing or empty names, two text values in attendance ("85%", "abc"), one missing attendance, two values outside 0-100 (120, -3), and two invalid programmes ("MECH" and NA).
- Unintended type conversion is detected because attendance is tested with as.numeric() and any non-missing value that becomes NA is reported as text.

Final representation: the core data frame with character IDs, names and programme, numeric attendance and a logical eligible column. The experiment, not preference, selected it.

TASK 3 - Create a Coercion-Safe Data Preparation Module (4 Marks)

Concept. When types are mixed in c(), R converts everything to the highest type in the order logical, integer, numeric, character. Some conversions are harmless (integer to numeric), some change meaning (TRUE to 1) and some destroy it (numbers to text). The module records the result type of each combination and classifies its risk.

- detect_coercion(...) records input types, the resulting type and a risk label.
- safe_convert(x, to) converts a vector and reports the failing positions, the number of NA values and lossy integer truncation.
- strict_combine(...) refuses to combine incompatible types (integer with numeric is allowed because it loses nothing).
- check_frame_types(df, schema) compares the classes of an imported data frame with the declared schema.

R code

```
# TASK 3: Coercion-safe data preparation module
options(width = 85)

detect_coercion <- function(...) {
  items    <- list(...)
  in_types <- vapply(items, function(i) class(i)[1], character(1))
  out_type <- class(do.call(c, items))[1]
  risk <- if (all(in_types == out_type)) "none"
          else if (out_type == "character") "HARMFUL: text"
          else if ("logical" %in% in_types) "semantic: TRUE->1"
          else "benign widening"
  data.frame(inputs = paste(in_types, collapse = " + "), output = out_type,
             risk = risk, stringsAsFactors = FALSE)
}

cat("--- Recorded result types of mixed inputs ---\n")
print(rbind(detect_coercion(1L, 2L),
            detect_coercion(1L, 2.5),
            detect_coercion(85, TRUE),
            detect_coercion(3L, FALSE),
            detect_coercion(85, "A"),
            detect_coercion(TRUE, "Pass"),
            detect_coercion(1L, 2.5, TRUE, "x")), right = FALSE)
```

```
safe_convert <- function(x, to = c("numeric", "integer", "logical", "character")) {
  to <- match.arg(to)
  was_na <- is.na(x)
  out <- suppressWarnings(switch(to, numeric = as.numeric(x),
                                 integer = as.integer(x),
                                 logical = as.logical(x),
                                 character = as.character(x)))
  failed <- which(is.na(out) & !was_na)
  lossy <- integer(0)
  if (to == "integer") {
    as_num <- suppressWarnings(as.numeric(x))
    lossy  <- which(!is.na(out) & !is.na(as_num) & as_num != out)
  }
  list(value = out, ok = length(failed) == 0 && length(lossy) == 0,
       failed_index = failed, lossy_index = lossy, n_missing = sum(is.na(out)))
}

strict_combine <- function(...) {
  items <- list(...)
  types <- unique(vapply(items, function(i) class(i)[1], character(1)))
  if (all(c("integer", "numeric") %in% types)) types <- setdiff(types, "integer")
  if (length(types) > 1)
    stop("mixed types refused: ", paste(types, collapse = ", "), call. = FALSE)
  do.call(c, items)
}

check_frame_types <- function(df, schema) {
  actual <- vapply(df[names(schema)], function(v) class(v)[1], character(1))
  bad <- actual != unlist(schema)
  data.frame(column = names(schema), expected = unlist(schema),
             actual = actual, mismatch = bad, row.names = NULL)
}

cat("\n--- Test cases for safe_convert ---\n")
cases <- list(valid_numeric = c("85", "90.5"), invalid_text = c("85", "abc", "70"),
```

```
            missing_value = c("85", NA, "70"), percent_sign = c("85%", "90"),
            truncation_risk = c("3.7", "4"), logical_words = c("TRUE", "yes", "T", "0"))
targets <- c("numeric", "numeric", "numeric", "numeric", "integer", "logical")
res <- do.call(rbind, Map(function(nm, x, to) {
  r <- safe_convert(x, to)
  data.frame(case = nm, target = to, accepted = r$ok,
             failed_at = paste(r$failed_index, collapse = ","),
             lossy_at = paste(r$lossy_index, collapse = ","),
             n_NA = r$n_missing)
}, names(cases), cases, targets))
print(res, row.names = FALSE)

cat("\n--- strict_combine versus c() ---\n")
cat("c(85, 'A')         ->", class(c(85, "A")), "(silent)\n")
r1 <- tryCatch(strict_combine(85, "A"), error = function(e) conditionMessage(e))
cat("strict_combine(85, 'A')   ->", r1, "\n")
cat("strict_combine(1L, 2.5)   ->", strict_combine(1L, 2.5), "(widening allowed)\n")
r3 <- tryCatch(strict_combine(85, TRUE), error = function(e) conditionMessage(e))
cat("strict_combine(85, TRUE)  ->", r3, "\n")

cat("\n--- check_frame_types on a file-style import ---\n")
raw <- read.csv(text = "reg_no,attendance,eligible
192324001,92,TRUE
192324002,85%,TRUE
192324003,74,FALSE", stringsAsFactors = FALSE)
schema_t <- list(reg_no = "character", attendance = "numeric", eligible = "logical")
print(check_frame_types(raw, schema_t))
cat("reg_no was read as", class(raw$reg_no), "-> leading zeros would be lost;",
    "attendance read as", class(raw$attendance), "because of '%'.\n")
```

Output (captured from running the code above)

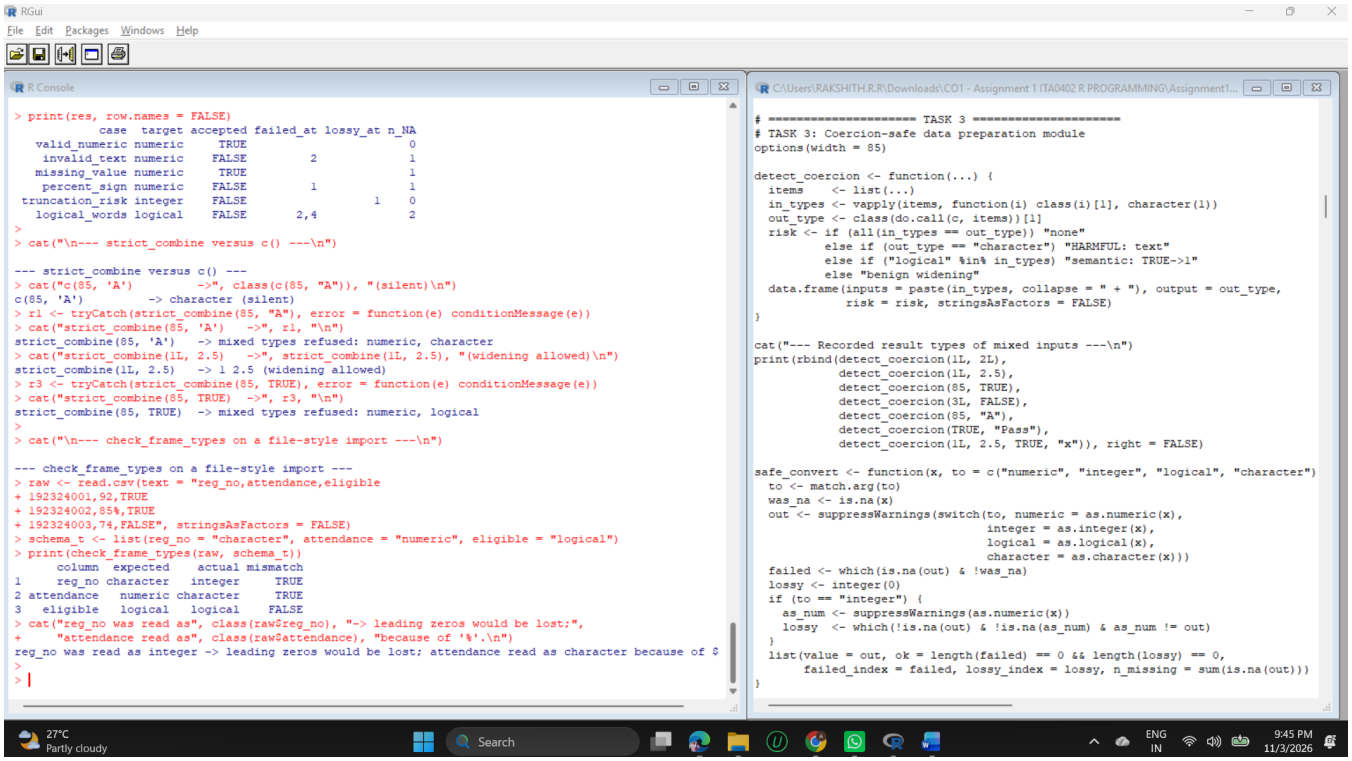
```
--- Recorded result types of mixed inputs ---
  inputs                         output    risk
1 integer + integer                 integer   none
2 integer + numeric                   numeric   benign widening
```

```
3 numeric + logical                    numeric   semantic: TRUE->1
4 integer + logical                   integer   semantic: TRUE->1
5 numeric + character                   character HARMFUL: text
6 logical + character                  character HARMFUL: text
7 integer + numeric + logical + character character HARMFUL: text


--- Test cases for safe_convert ---
           case  target accepted failed_at lossy_at n_NA
   valid_numeric numeric     TRUE                    0
    invalid_text numeric    FALSE        2           1
   missing_value numeric     TRUE                    1
    percent_sign numeric    FALSE        1           1
 truncation_risk integer    FALSE               1    0
   logical_words logical    FALSE       2,4          2


--- strict_combine versus c() ---
c(85, 'A')         -> character (silent)
strict_combine(85, 'A')   -> mixed types refused: numeric, character
strict_combine(1L, 2.5)   -> 1 2.5 (widening allowed)
strict_combine(85, TRUE)  -> mixed types refused: numeric, logical


--- check_frame_types on a file-style import ---
      column  expected    actual mismatch
1     reg_no character   integer     TRUE
2 attendance   numeric character     TRUE
3   eligible   logical   logical    FALSE
reg_no was read as integer -> leading zeros would be lost; attendance read as character because of '%'.
```



Interpretation of the experiments

- **Recorded types:** integer plus numeric gives numeric (benign); numeric or integer plus logical absorbs TRUE/FALSE as numbers (semantic change); anything plus character gives character (harmful).
- **Valid, invalid, missing, mixed cases:** valid_numeric is accepted; invalid_text fails at position 2; missing_value is accepted with one NA, because a genuine missing value is not a conversion failure; percent_sign fails at position 1; truncation_risk ("3.7" to integer) is flagged as lossy at position 1; logical_words fails at positions 2 and 4 because "yes" and "0" are not recognised by as.logical().
- **Prevention:** c(85, "A") silently returned character, while strict_combine(85, "A") and strict_combine(85, TRUE) stopped with a clear message.
- **Import check:** read.csv() read reg_no as integer, which would drop leading zeros of real IDs, and read attendance as character because of the % sign. check_frame_types() exposed both mismatches before analysis.

How semantic meaning is preserved. The module uses only native vectors and data frames. Meaning is protected by rules around them: identifiers stay character, TRUE/FALSE never become 1/0 inside a mixed vector without notice, widening is allowed only when lossless, and every failed conversion is reported with its position instead of becoming a silent NA.

TASK 4 - Construct a Student-by-Subject Matrix Processing Engine (4 Marks)

Concept. A matrix is a vector with a dimension attribute, so every cell has the same type. That gives fast row and column summaries and linear algebra, but a matrix has no room for metadata. Row and column names (dimnames) therefore carry the student and subject identifiers, and a data frame carries everything else.

- 12 students (from core) by 5 subjects (CS101 to EN105) with credits 4, 4, 3, 4, 2.
- Two marks are deliberately below 40 (EN105 for 192324006 and DS104 for 192324007) so that pass and fail logic is visible.
- Matrix operations used: %*% for a credit-weighted average, t() for transposition, cor() for subject relationships.

R code

```
# TASK 4: Student-by-subject matrix processing engine
options(width = 85)
set.seed(2026)

subjects <- data.frame(code = c("CS101", "MA102", "ST103", "DS104", "EN105"),
                       title = c("Programming", "Mathematics", "Statistics",
                                 "Data Science", "Communication"),
                       credits = c(4, 4, 3, 4, 2), stringsAsFactors = FALSE)
ability <- rnorm(12, 68, 9)
marks <- sapply(seq_len(5), function(j) round(pmin(100, pmax(0, ability + rnorm(12, 0, 8)))))
dimnames(marks) <- list(core$reg_no, subjects$code)
marks["192324006", "EN105"] <- 38   # deliberate failing marks so pass/fail logic is visible
marks["192324007", "DS104"] <- 36
cat("Marks matrix (students x subjects), dim =", dim(marks), "\n")
print(marks)

cat("\n--- Indexing ---\n")
cat("By position  marks[3, 2]                :", marks[3, 2], "\n")
cat("By names     marks['192324003','MA102']  :", marks["192324003", "MA102"], "\n")
cat("One student  marks['192324005', ]:\n"); print(marks["192324005", ])
cat("One subject, first 4 students  marks[1:4, 'ST103']:\n"); print(marks[1:4, "ST103"])
cat("Sub-matrix kept as matrix (drop = FALSE):\n")
print(marks[1:3, c("CS101", "DS104"), drop = FALSE])
```

```
cat("\n--- Row and column summaries ---\n")
cat("Student averages (rowMeans):\n"); print(round(rowMeans(marks), 2))
cat("Subject averages (colMeans):\n"); print(round(colMeans(marks), 2))
cat("Subject standard deviations:\n"); print(round(apply(marks, 2, sd), 2))
cat("Subjects passed per student (>= 40):\n"); print(rowSums(marks >= 40))

cat("\n--- Transposition ---\n")
tm <- t(marks)
cat("dim(t(marks)) =", dim(tm), " | dimnames swapped:",
    identical(dimnames(tm), rev(dimnames(marks))), "\n")
print(tm[, 1:4])

cat("\n--- Matrix operation 1: credit-weighted average (matrix %*% vector) ---\n")
w <- subjects$credits / sum(subjects$credits)
weighted_avg <- drop(marks %*% w)
names(weighted_avg) <- rownames(marks)
print(round(weighted_avg, 2))
cat("Top 3 students:", paste(names(sort(weighted_avg, decreasing = TRUE))[1:3],
                             collapse = ", "), "\n")

cat("\n--- Matrix operation 2: subject correlation matrix ---\n")
print(round(cor(marks), 2))

cat("\n--- Linking matrix positions to identifiers ---\n")
hit <- which(marks == max(marks), arr.ind = TRUE)[1, ]
cat("Highest mark", max(marks), "-> row", hit[1], "col", hit[2], "=>",
    rownames(marks)[hit[1]], "in", colnames(marks)[hit[2]],
    "(", subjects$title[match(colnames(marks)[hit[2]], subjects$code)], ")\n")
locate_mark <- function(m, id, subject) {
  r <- match(id, rownames(m)); c <- match(subject, colnames(m))
  if (is.na(r) || is.na(c)) stop("unknown ID or subject: ", id, " / ", subject)
  list(row = r, col = c, mark = m[r, c])
}
str(locate_mark(marks, "192324007", "EN105"))
cat("Top scorer per subject:\n")
print(apply(marks, 2, function(col) rownames(marks)[which.max(col)]))
```

```
cat("\n--- Matrix -> data frame (long, metadata-rich) ---\n")
long <- as.data.frame(as.table(marks), stringsAsFactors = FALSE)
names(long) <- c("reg_no", "subject", "mark")
long <- merge(long, core[, c("reg_no", "programme", "attendance")], by = "reg_no")
long <- merge(long, subjects[, c("code", "title", "credits")],
              by.x = "subject", by.y = "code")
long <- long[order(long$reg_no, match(long$subject, subjects$code)), ]
rownames(long) <- NULL
cat("rows:", nrow(long), "(12 students x 5 subjects)\n"); print(head(long, 6))

cat("\n--- Data frame -> matrix (round trip) ---\n")
back <- tapply(long$mark, list(long$reg_no, long$subject), sum)
back <- back[rownames(marks), colnames(marks)]
cat("After tapply(), columns come back alphabetical:",
    paste(colnames(tapply(long$mark, list(long$reg_no, long$subject), sum)), collapse = " "), "\n")
cat("After re-indexing by name, identical to original:",
    isTRUE(all.equal(back, marks, check.attributes = FALSE)) &&
      identical(dimnames(back), dimnames(marks)), "\n")

png("fig/fig_t4_heatmap.png", width = 1500, height = 1000, res = 170)
par(mar = c(5, 7, 4, 2))
image(1:5, 1:12, t(marks)[, 12:1], col = hcl.colors(25, "YlGnBu", rev = TRUE),
      axes = FALSE, xlab = "Subject", ylab = "")
axis(1, 1:5, colnames(marks)); axis(2, 1:12, rev(rownames(marks)), las = 1, cex.axis = 0.8)
for (i in 1:12) for (j in 1:5)
  text(j, 13 - i, marks[i, j], cex = 0.8, col = if (marks[i, j] >= 68) "white" else "black")
title("Student-by-subject marks matrix (interpretable via dimnames)")
dev.off()
```

Output (captured from running the code above)

```
Marks matrix (students x subjects), dim = 12 5
          CS101 MA102 ST103 DS104 EN105
192324001    71    73    77    79    72
192324002    56    74    51    62    60
```

```
192324003    49    83    74    60    68
192324004    78    68    61    66    65
192324005    67    67    53    55    48
192324006    47    59    52    49    38
192324007    55    57    52    36    47
192324008    64    60    61    57    68
192324009    66    67    62    59    61
192324010    62    66    75    56    65
192324011    53    66    70    65    68
192324012    73    71    58    53    61

--- Indexing ---
By position  marks[3, 2]                : 83
By names     marks['192324003','MA102']  : 83
One student  marks['192324005', ]:
CS101 MA102 ST103 DS104 EN105
  67    67    53    55    48
One subject, first 4 students  marks[1:4, 'ST103']:
192324001 192324002 192324003 192324004
       77        51        74        61
Sub-matrix kept as matrix (drop = FALSE):
          CS101 DS104
192324001    71    79
192324002    56    62
192324003    49    60

--- Row and column summaries ---
Student averages (rowMeans):
192324001 192324002 192324003 192324004 192324005 192324006 192324007 192324008
    74.4      60.6      66.8      67.6      58.0      49.0      49.4      62.0
192324009 192324010 192324011 192324012
    63.0      64.8      64.4      63.2
Subject averages (colMeans):
CS101 MA102 ST103 DS104 EN105
61.75 67.58 62.17 58.08 60.08
Subject standard deviations:
```

```
CS101 MA102 ST103 DS104 EN105
 9.83  7.19  9.64 10.35 10.38
Subjects passed per student (>= 40):
192324001 192324002 192324003 192324004 192324005 192324006 192324007 192324008
        5         5         5         5         5         4         4         5
192324009 192324010 192324011 192324012
        5         5         5         5

--- Transposition ---
dim(t(marks)) = 5 12  | dimnames swapped: TRUE
       192324001 192324002 192324003 192324004
CS101         71        56        49        78
MA102         73        74        83        68
ST103         77        51        74        61
DS104         79        62        60        66
EN105         72        60        68        65

--- Matrix operation 1: credit-weighted average (matrix %*% vector) ---
192324001 192324002 192324003 192324004 192324005 192324006 192324007 192324008
    74.53     61.24     66.24     68.29     59.47     50.12     49.53     61.35
192324009 192324010 192324011 192324012
    63.29     64.18     63.65     63.76
Top 3 students: 192324001, 192324004, 192324003

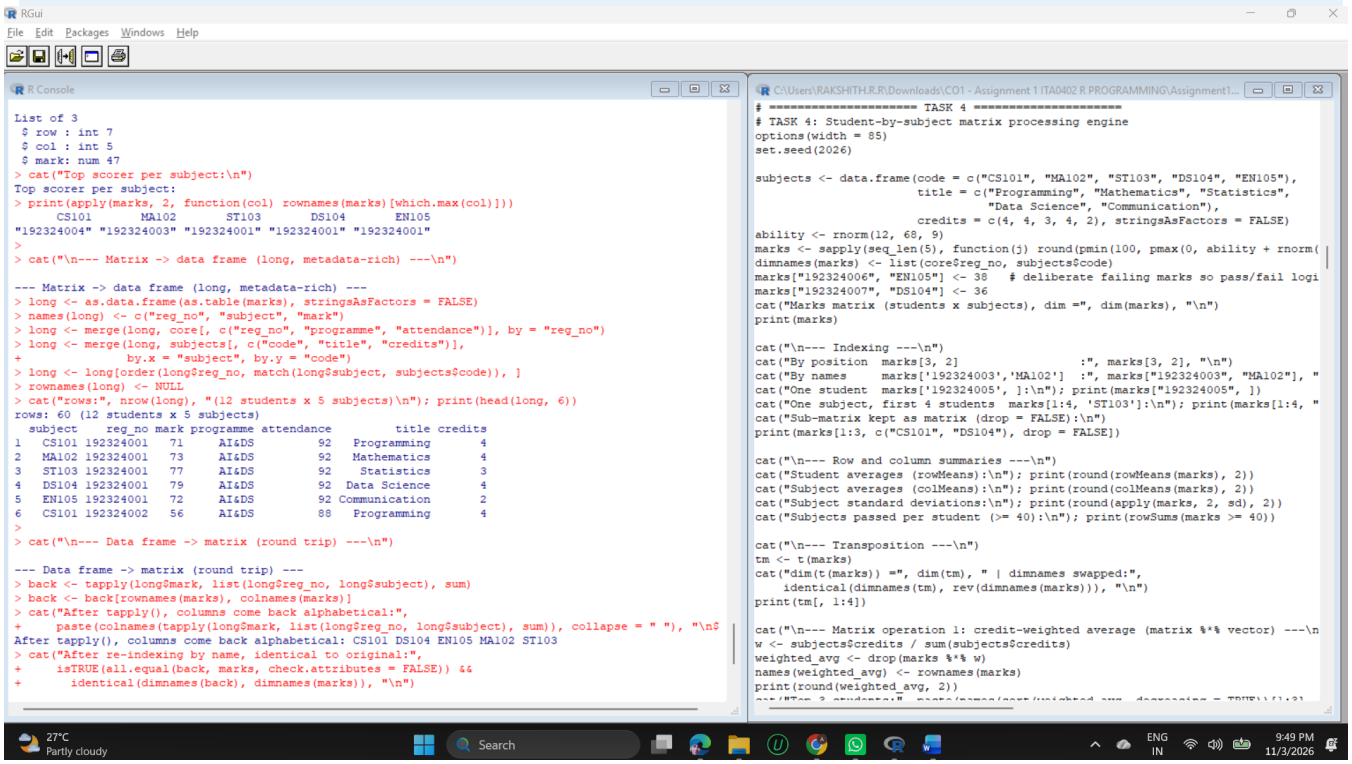
--- Matrix operation 2: subject correlation matrix ---
       CS101 MA102 ST103 DS104 EN105
CS101  1.00  0.07  0.08  0.38  0.36
MA102  0.07  1.00  0.46  0.57  0.54
ST103  0.08  0.46  1.00  0.61  0.77
DS104  0.38  0.57  0.61  1.00  0.73
EN105  0.36  0.54  0.77  0.73  1.00

--- Linking matrix positions to identifiers ---
Highest mark 83 -> row 3 col 2 => 192324003 in MA102 ( Mathematics )
List of 3
 $ row : int 7
```

```
$ col : int 5
$ mark: num 47
Top scorer per subject:
     CS101       MA102       ST103       DS104       EN105
"192324004" "192324003" "192324001" "192324001" "192324001"

--- Matrix -> data frame (long, metadata-rich) ---
rows: 60 (12 students x 5 subjects)
  subject    reg_no mark programme attendance         title credits
1   CS101 192324001   71     AI&DS         92   Programming       4
2   MA102 192324001   73     AI&DS         92   Mathematics       4
3   ST103 192324001   77     AI&DS         92    Statistics       3
4   DS104 192324001   79     AI&DS         92  Data Science       4
5   EN105 192324001   72     AI&DS         92 Communication       2
6   CS101 192324002   56     AI&DS         88   Programming       4

--- Data frame -> matrix (round trip) ---
After tapply(), columns come back alphabetical: CS101 DS104 EN105 MA102 ST103
After re-indexing by name, identical to original: TRUE
```



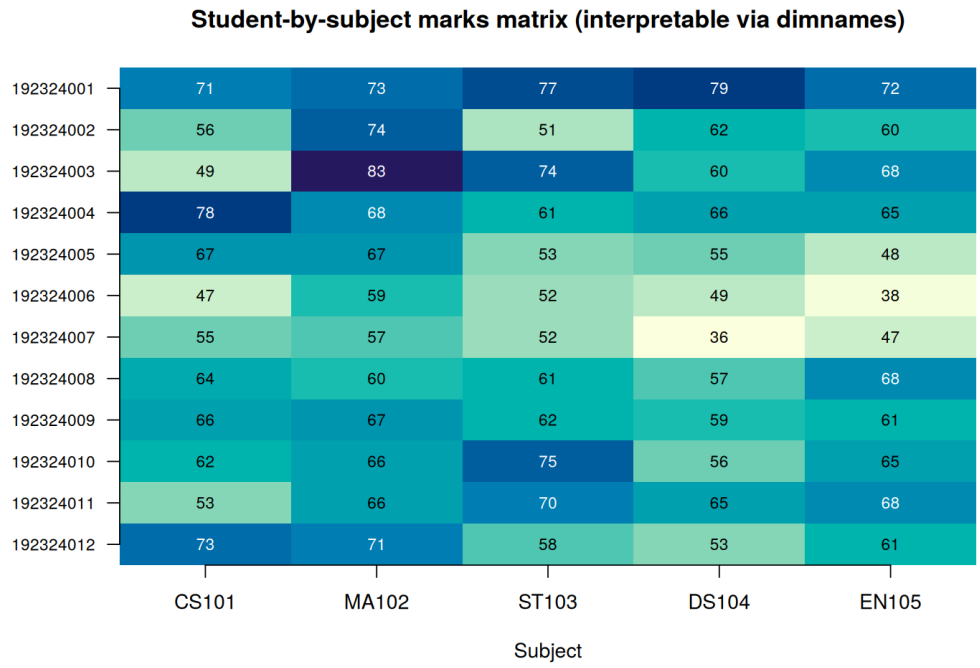


Figure 2: Marks matrix drawn as a heat map; the axes come directly from the matrix dimnames

Results and interpretation

- **Indexing** by position and by name returned the same cell (83 for 192324003 in MA102). Name-based indexing is the safer form because it survives re-ordering.
- **Summaries:** subject means range from 58.08 (DS104) to 67.58 (MA102). Students 192324006 and 192324007 each passed 4 of 5 subjects; all others passed 5.
- **Credit-weighted average (`marks %*% weights`):** the three highest students are 192324001, 192324004 and 192324003. Weighting by credits gives programming and mathematics more influence than communication.
- **Correlation:** ST103 and EN105 correlate most strongly (0.77), CS101 is almost independent of MA102 and ST103 (0.07 and 0.08).
- **Position to identifier mapping:** which(..., arr.ind = TRUE) returns row 3, column 2, and dimnames turns that into student 192324003 in MA102 (Mathematics). locate_mark() does the same lookup by ID and subject and stops on an unknown ID.
- **Conversion strategy:** matrix to long data frame with as.data.frame(as.table()), then merge() attaches programme, attendance, subject title and credits (60 rows = 12 x 5). The way back uses tapply(), which returned the columns alphabetically (CS101 DS104 EN105 MA102 ST103). Re-indexing by names restored the original order and the result was identical to the starting matrix.

Justification. The matrix is kept for numeric work. The long data frame is used whenever grouping, filtering or modelling needs metadata. Conversions go through names, not positions, so interpretability is not lost.

TASK 5 - Create a Multi-Dimensional Assessment Array Model (4 Marks)

Concept. An array extends a matrix to more dimensions. The array has dimensions student x subject x component, and each dimension is named twice: by the dimension's own name (student, subject, component) and by its labels. Retrieval and calculation then read like the question being asked.

- Dimensions: 12 students, 5 subjects, 3 components (internal, model, practical).
- The model slice is the Task 4 matrix, so the two components share one source of truth.
- Weights 30% internal, 30% model, 40% practical produce the weighted total.

R code

```
# TASK 5: Multi-dimensional assessment array (students x subjects x components)
options(width = 85)
set.seed(77)
components <- c("internal", "model", "practical")
arr <- array(NA_real_, dim = c(12, 5, 3),
             dimnames = list(student = rownames(marks), subject = colnames(marks),
                             component = components))
arr[, , "model"]     <- marks                       # reuse the Task 4 matrix
arr[, , "internal"]  <- round(pmin(100, pmax(0, marks + rnorm(60, 4, 6))))
arr[, , "practical"] <- round(pmin(100, pmax(0, marks + rnorm(60, 8, 7))))
cat("dim(arr) =", dim(arr), "| named dimensions:", paste(names(dimnames(arr)), collapse = ", "), "\n")
cat("Model slice equals the Task 4 matrix (values and IDs):",
    isTRUE(all.equal(arr[, , "model"], marks, check.attributes = FALSE)) &&
      identical(rownames(arr), rownames(marks)) && identical(colnames(arr), colnames(marks)), "\n")
cat("\nFirst two students (all subjects, all components):\n")
print(arr[1:2, , ])

cat("\n--- Retrieval ---\n")
cat("One cell     arr['192324003','ST103','internal'] :", arr["192324003", "ST103", "internal"], "\n")
cat("One student/subject across components:\n"); print(arr["192324003", "ST103", ])
cat("All students, CS101 practical (first 5):", arr[1:5, "CS101", "practical"], "\n")
sub <- arr[1:3, c("CS101", "MA102"), c("internal", "practical")]
cat("Sub-array dims:", dim(sub), "| dimnames kept:",
    paste(names(dimnames(sub)), collapse = ", "), "\n")
```

```
cat("\n--- Calculations over selected dimensions ---\n")
cat("Mean mark per component (all students and subjects):\n"); print(round(apply(arr, 3, mean), 2))
cat("Mean per subject x component:\n"); print(round(apply(arr, c(2, 3), mean), 1))
cat("Overall mean per student (first 5):\n"); print(round(apply(arr, 1, mean)[1:5], 2))
w <- c(internal = 0.3, model = 0.3, practical = 0.4)
total <- Reduce(`+`, lapply(components, function(k) arr[, , k] * w[k]))
total_apply <- apply(arr, c(1, 2), function(x) sum(x * w))
cat("Weighted total (30/30/40) via vectorised sum equals apply():",
    isTRUE(all.equal(total, total_apply)), "\n")
print(round(total[1:4, ], 1))

cat("\n--- Why dimensions stay interpretable ---\n")
p <- aperm(arr, c(2, 1, 3))
cat("aperm(arr, c(2,1,3)) dim:", dim(p), "| names:", paste(names(dimnames(p)), collapse = ", "), "\n")
cat("Same cell by names after permutation:", p["ST103", "192324003", "internal"] ==
      arr["192324003", "ST103", "internal"], "\n")
e1 <- tryCatch(arr["XXXX", "CS101", "internal"], error = function(e) conditionMessage(e))
e2 <- tryCatch(arr[1, 1, 4], error = function(e) conditionMessage(e))
cat("Unknown ID    ->", e1, "\n"); cat("Component 4   ->", e2, "\n")

cat("\n--- Transformations ---\n")
long5 <- as.data.frame.table(arr, responseName = "mark", stringsAsFactors = FALSE)
cat("1) array -> long data frame:", nrow(long5), "rows x", ncol(long5), "cols\n")
print(head(long5, 4))
by_comp <- lapply(setNames(components, components), function(k) arr[, , k])
cat("2) array -> list of matrices:", paste(names(by_comp), sapply(by_comp, function(m) paste(dim(m), collapse
= "x"))), "\n")
report <- data.frame(reg_no = rownames(total), round(total, 1), check.names = FALSE, row.names = NULL)
cat("3) array -> wide report of weighted totals:\n"); print(head(report, 4))
back <- xtabs(mark ~ student + subject + component, data = long5)
cat("4) long -> array round trip identical:",
    isTRUE(all.equal(unclass(back)[rownames(arr), colnames(arr), components],
                     arr, check.attributes = FALSE)), "\n")

png("fig/fig_t5_components.png", width = 1500, height = 950, res = 170)
par(mar = c(5, 5, 4, 2))
```

```
boxplot(mark ~ component, data = long5, col = c("#9ecae1", "#a1d99b", "#fdae6b"),
        ylab = "Marks", xlab = "Assessment component",
        main = "Marks by assessment component (array dimension 3)")
points(1:3, apply(arr, 3, mean), pch = 18, cex = 1.6, col = "red")
legend("bottomright", legend = "Mean", pch = 18, col = "red", bty = "n")
dev.off()
```

Output (captured from running the code above)

```
dim(arr) = 12 5 3 | named dimensions: student, subject, component
Model slice equals the Task 4 matrix (values and IDs): TRUE

First two students (all subjects, all components):
, , component = internal

           subject
student     CS101 MA102 ST103 DS104 EN105
  192324001    72    76    72    84    78
  192324002    67    78    56    65    67

, , component = model

           subject
student     CS101 MA102 ST103 DS104 EN105
  192324001    71    73    77    79    72
  192324002    56    74    51    62    60

, , component = practical

           subject
student     CS101 MA102 ST103 DS104 EN105
  192324001    82    84    84    84    75
  192324002    64    84    62    77    71



--- Retrieval ---
```

```
One cell     arr['192324003','ST103','internal'] : 74
One student/subject across components:
 internal     model practical
       74        74        83
All students, CS101 practical (first 5): 82 64 50 86 74
Sub-array dims: 3 2 2 | dimnames kept: student, subject, component

--- Calculations over selected dimensions ---
Mean mark per component (all students and subjects):
 internal     model practical
    66.08     61.93     69.47
Mean per subject x component:
       component
subject internal model practical
  CS101     66.2  61.8      69.2
  MA102     75.2  67.6      76.2
  ST103     64.0  62.2      67.8
  DS104     61.9  58.1      65.8
  EN105     63.1  60.1      68.2
Overall mean per student (first 5):
192324001 192324002 192324003 192324004 192324005
    77.53     66.27     71.33     73.80     63.20
Weighted total (30/30/40) via vectorised sum equals apply(): TRUE
           subject
student     CS101 MA102 ST103 DS104 EN105
  192324001  75.7  78.3  78.3  82.5  75.0
  192324002  62.5  79.2  56.9  68.9  66.5
  192324003  51.8  87.2  77.6  71.2  72.0
  192324004  84.2  80.0  65.0  71.5  71.3

--- Why dimensions stay interpretable ---
aperm(arr, c(2,1,3)) dim: 5 12 3 | names: subject, student, component
Same cell by names after permutation: TRUE
Unknown ID    -> subscript out of bounds
Component 4   -> subscript out of bounds
```

--- Transformations ---

1) array -> long data frame: 180 rows x 4 cols

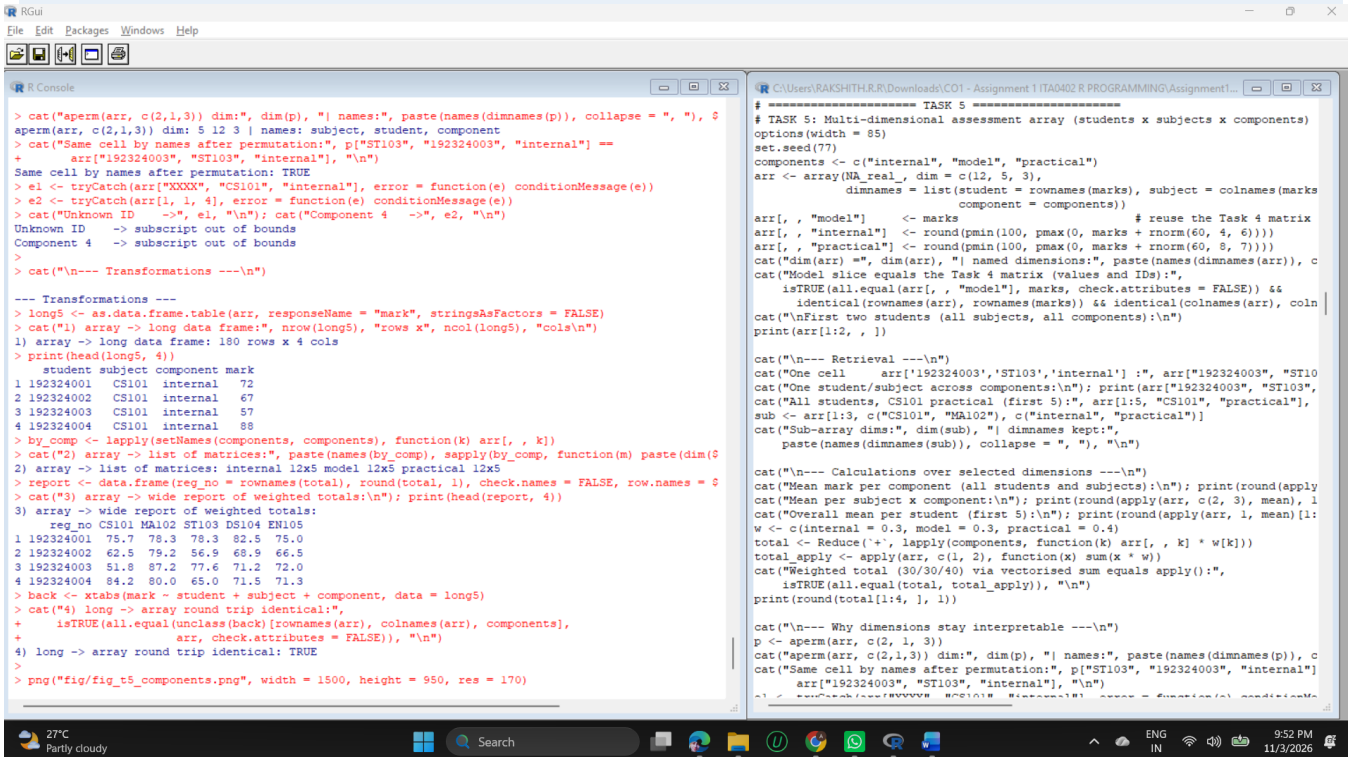
	student	subject	component	mark
1	192324001	CS101	internal	72
2	192324002	CS101	internal	67
3	192324003	CS101	internal	57
4	192324004	CS101	internal	88

2) array -> list of matrices: internal 12x5 model 12x5 practical 12x5

3) array -> wide report of weighted totals:

	reg_no	CS101	MA102	ST103	DS104	EN105
1	192324001	75.7	78.3	78.3	82.5	75.0
2	192324002	62.5	79.2	56.9	68.9	66.5
3	192324003	51.8	87.2	77.6	71.2	72.0
4	192324004	84.2	80.0	65.0	71.5	71.3

4) long -> array round trip identical: TRUE



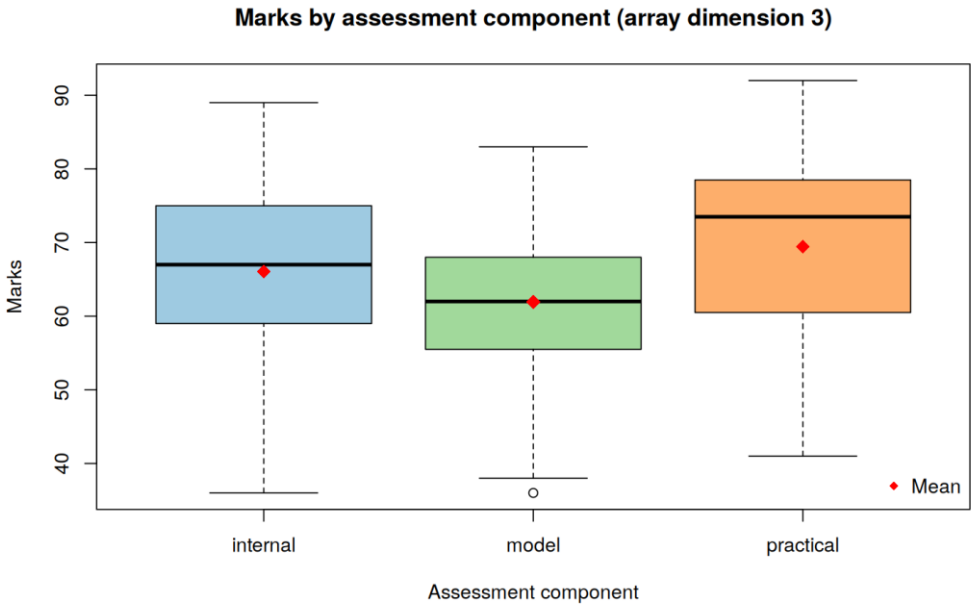


Figure 3: Marks by assessment component; red diamonds show the means

Results and interpretation

- **Retrieval:** one cell, one student-subject across components, one subject column, and a sub-array of 3 x 2 x 2 that kept all dimension names.
- **Calculation:** mean mark per component is 66.08 (internal), 61.93 (model) and 69.47 (practical). Practical marks are highest, model examination marks lowest. apply(arr, c(2, 3), mean) gives the subject-by-component table.
- **Weighted total:** a vectorised sum over components equals the slower apply() version, so the faster form is safe to use.
- **Interpretability:** aperm() reordered the dimensions to subject, student, component and the names moved with the data; the same cell was found by name before and after. Wrong labels fail loudly: an unknown ID and a fourth component both gave subscript out of bounds.
- **Transformations:** (1) long data frame of 180 rows (12 x 5 x 3); (2) list of three matrices; (3) wide report of weighted totals; (4) long frame back to an array, identical to the original.

Justification. The array keeps three related measures aligned by name. A list of matrices or a long data frame is produced only when a report or model needs that shape.

TASK 6 - Build an Integrated List, Factor and Data-Frame Layer (5 Marks)

Concept. A list holds one heterogeneous record. Factors store categorical variables as integer codes with a fixed set of levels. A data frame is a list of equal-length columns, which makes it the natural analysis table and the meeting point for vectors, factors and numbers.

- List record: identifiers, marks vector, logical eligibility and a nested contact list.
- Factors: programme (4 levels) and an ordered grade factor F < C < B < A < O made with cut().
- Operations written as functions: add, update, select and summarise.
- recommend_type() is the rule that decides between character and factor.

R code

```
# TASK 6: Integrated list, factor and data-frame layer
options(width = 85)
rec <- list(reg_no = "192324003", name = "Charan Teja", programme = "CSE",
            attendance = 74.5, marks = c(CS101 = 71, MA102 = 66, ST103 = 59),
            eligible = FALSE, contact = list(email = "charan@example.edu", phone = "9876500003"))
cat("--- Heterogeneous student record (list) ---\n"); str(rec)
cat("Nested access rec$contact$email:", rec$contact$email, "| rec[['marks']]['ST103']:",
rec[["marks"]]["ST103"], "\n")
cat("Element classes:", paste(names(rec), sapply(rec, class), sep = "=", collapse = "; "), "\n")

grade_breaks <- c(-Inf, 40, 55, 70, 85, Inf); grade_labels <- c("F", "C", "B", "A", "O")
avg_total <- rowMeans(total)
analysis <- data.frame(
  reg_no     = core$reg_no, name = core$name,
  programme  = factor(core$programme, levels = valid_programmes),
  attendance = core$attendance, eligible = core$eligible,
  avg_total  = round(unname(avg_total[core$reg_no]), 2),
  grade      = cut(avg_total[core$reg_no], breaks = grade_breaks, labels = grade_labels,
                   right = FALSE, ordered_result = TRUE),
  stringsAsFactors = FALSE)
cat("\n--- Analysis-ready data frame ---\n"); str(analysis); print(analysis[1:5, ])
cat("\nlevels(programme):", levels(analysis$programme), "\n")
cat("levels(grade) [ordered]:", paste(levels(analysis$grade), collapse = " < "), "\n")
```

```
cat("\n--- Rule: character or factor? ---\n")
recommend_type <- function(x, max_levels = 20, max_unique_ratio = 0.5) {
  n <- length(x); u <- length(unique(x[!is.na(x)]))
  if (!is.character(x)) return("keep as is")
  if (u / n > max_unique_ratio || u > max_levels) "character" else "factor"
}
decision <- data.frame(variable = c("reg_no", "name", "programme", "gender", "email_domain"),
  unique_values = c(12, 12, 4, 2, 3),
  decision = c(recommend_type(core$reg_no), recommend_type(core$name),
               recommend_type(core$programme),
               recommend_type(rep(c("F", "M"), 6)),
               recommend_type(rep(c("a.edu", "b.edu", "c.edu"), 4))))
print(decision, row.names = FALSE)

cat("\n--- Operations: add, update, select, summarise ---\n")
add_student <- function(df, reg_no, name, programme, attendance, avg_total) {
  if (!programme %in% levels(df$programme)) stop("unknown programme: ", programme)
  new <- data.frame(reg_no = reg_no, name = name,
                    programme = factor(programme, levels = levels(df$programme)),
                    attendance = attendance, eligible = attendance >= 75,
                    avg_total = avg_total,
                    grade = cut(avg_total, grade_breaks, grade_labels, right = FALSE, ordered_result = TRUE),
                    stringsAsFactors = FALSE)
  rbind(df, new)
}
analysis <- add_student(analysis, "192324013", "Meena K", "IT", 91, 73.4)
cat("ADD    rows now:", nrow(analysis), "| last row:", analysis$name[13], as.character(analysis$grade[13]), "\n")
bad <- tryCatch(add_student(analysis, "192324014", "X", "MECH", 80, 60), error = function(e) conditionMessage(e))
cat("ADD (invalid programme) ->", bad, "\n")
analysis$attendance[analysis$reg_no == "192324003"] <- 76
analysis$eligible <- analysis$attendance >= 75
cat("UPDATE 192324003 attendance -> 76, eligible =", analysis$eligible[analysis$reg_no == "192324003"], "\n")
cat("SELECT grade >= 'A' (ordered factor comparison):\n")
print(analysis[analysis$grade >= "A", c("reg_no", "programme", "avg_total", "grade")])
```

```
cat("SUMMARISE mean avg_total by programme:\n")
print(aggregate(avg_total ~ programme, data = analysis, FUN = function(x) round(mean(x), 2)))
cat("SUMMARISE programme x grade table:\n"); print(table(analysis$programme, analysis$grade))

cat("\n--- Exchange with matrix and array components ---\n")
tot_df <- data.frame(reg_no = rownames(total), total, row.names = NULL)
back_m <- as.matrix(tot_df[, colnames(total)]); rownames(back_m) <- tot_df$reg_no
cat("matrix -> data frame -> matrix identical:",
    isTRUE(all.equal(back_m, total, check.attributes = FALSE)) &&
      identical(rownames(back_m), rownames(total)), "\n")
joined <- merge(analysis[, c("reg_no", "programme")], tot_df, by = "reg_no")
cat("Subject means by programme (data frame groups applied to matrix columns):\n")
sm <- aggregate(. ~ programme, data = joined[, -1], FUN = mean)
sm[-1] <- round(sm[-1], 1); print(sm)
pg <- with(analysis, table(programme, grade))
cat("data-frame summary -> 2-D table class:", class(pg), "dim:", dim(pg), "\n")
arr_back <- array(unlist(by_comp), dim = dim(arr), dimnames = dimnames(arr))
cat("list of matrices -> array identical:", isTRUE(all.equal(arr_back, arr)), "\n")

png("fig/fig_t6_grades.png", width = 1500, height = 950, res = 170)
par(mar = c(5, 5, 4, 2))
barplot(t(table(analysis$programme, analysis$grade)), beside = FALSE,
        col = hcl.colors(5, "Blues 3"), legend.text = TRUE, args.legend = list(title = "Grade", x = "topright"),
        xlab = "Programme (factor)", ylab = "Number of students",
        main = "Grade distribution by programme (ordered factor)")
dev.off()
```

Output (captured from running the code above)

```
--- Heterogeneous student record (list) ---
List of 7
 $ reg_no    : chr "192324003"
 $ name      : chr "Charan Teja"
 $ programme : chr "CSE"
 $ attendance: num 74.5
 $ marks     : Named num [1:3] 71 66 59
```

```
 ..- attr(*, "names")= chr [1:3] "CS101" "MA102" "ST103"
 $ eligible  : logi FALSE
 $ contact   :List of 2
  ..$ email: chr "charan@example.edu"
  ..$ phone: chr "9876500003"
Nested access rec$contact$email: charan@example.edu | rec[['marks']]['ST103']: 59
Element classes: reg_no=character; name=character; programme=character; attendance=numeric; marks=numeric; eligible=logical; contact=list

--- Analysis-ready data frame ---
'data.frame':    12 obs. of  7 variables:
 $ reg_no    : chr  "192324001" "192324002" "192324003" "192324004" ...
 $ name      : chr  "Aarav Kumar" "Bhavya Rao" "Charan Teja" "Divya Nair" ...
 $ programme : Factor w/ 4 levels "AI&DS","CSE",..: 1 1 2 3 4 1 2 3 1 4 ...
 $ attendance: num  92 88 74.5 96 81 69 90 75 85.5 78 ...
 $ eligible  : logi  TRUE TRUE FALSE TRUE TRUE FALSE ...
 $ avg_total : num  78 66.8 72 74.4 63.2 ...
 $ grade     : Ord.factor w/ 5 levels "F"<"C"<"B"<"A"<..: 4 3 4 4 3 2 2 3 3 3 ...
    reg_no          name programme attendance eligible avg_total grade
1 192324001    Aarav Kumar     AI&DS       92.0     TRUE     77.96     A
2 192324002    Bhavya Rao      AI&DS       88.0     TRUE     66.80     B
3 192324003    Charan Teja       CSE       74.5    FALSE     71.96     A
4 192324004    Divya Nair        IT        96.0     TRUE     74.40     A
5 192324005 Eshwar Pillai        ECE       81.0     TRUE     63.20     B

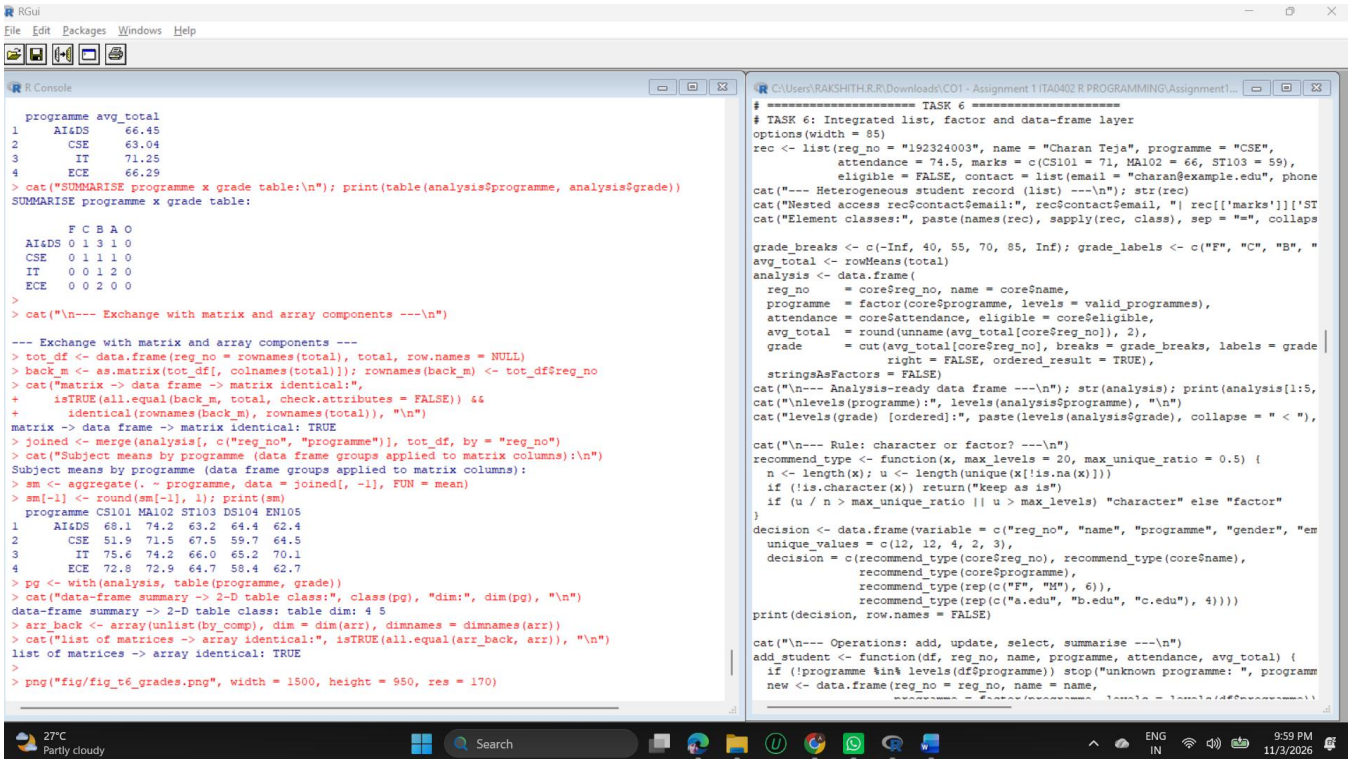
levels(programme): AI&DS CSE IT ECE
levels(grade) [ordered]: F < C < B < A < O

--- Rule: character or factor? ---
     variable unique_values  decision
       reg_no           12 character
         name           12 character
    programme            4    factor
       gender            2    factor
 email_domain            3    factor
```

```
--- Operations: add, update, select, summarise ---
ADD    rows now: 13 | last row: Meena K A
ADD (invalid programme) -> unknown programme: MECH
UPDATE 192324003 attendance -> 76, eligible = TRUE
SELECT grade >= 'A' (ordered factor comparison):
      reg_no programme avg_total grade
1  192324001     AI&DS     77.96     A
3  192324003       CSE     71.96     A
4  192324004        IT     74.40     A
13 192324013        IT     73.40     A
SUMMARISE mean avg_total by programme:
  programme avg_total
1     AI&DS     66.45
2       CSE     63.04
3        IT     71.25
4       ECE     66.29
SUMMARISE programme x grade table:

       F C B A O
  AI&DS 0 1 3 1 0
  CSE   0 1 1 1 0
  IT    0 0 1 2 0
  ECE   0 0 2 0 0

--- Exchange with matrix and array components ---
matrix -> data frame -> matrix identical: TRUE
Subject means by programme (data frame groups applied to matrix columns):
  programme CS101 MA102 ST103 DS104 EN105
1     AI&DS  68.1  74.2  63.2  64.4  62.4
2       CSE  51.9  71.5  67.5  59.7  64.5
3        IT  75.6  74.2  66.0  65.2  70.1
4       ECE  72.8  72.9  64.7  58.4  62.7
data-frame summary -> 2-D table class: table dim: 4 5
list of matrices -> array identical: TRUE
```



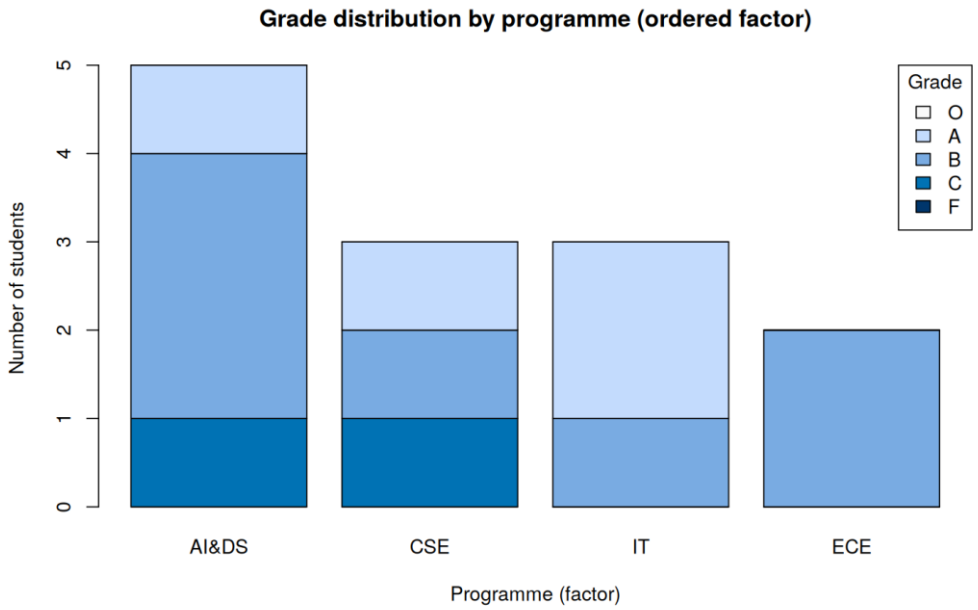


Figure 4: Grade distribution by programme using the ordered factor

Rules for character versus factor

Rule	Applied to	Decision
Mostly unique values or identifier-like (unique ratio above 0.5, or more than 20 distinct values)	reg_no, name	Keep as character

Few repeated labels from a controlled list	programme, grade, gender	Factor (ordered when the labels have a rank)
Numbers stored as text	any	Convert with as.numeric(as.character()), never keep as factor
Categories that will grow often	any	Character, plus validation against a list

Results and interpretation

- **Add:** a valid student increased the frame to 13 rows; adding programme "MECH" was refused by the function, so no NA level was created.
- **Update:** attendance of 192324003 was changed from 74.5 to 76 and eligible recalculated to TRUE.
- **Select:** because grade is an ordered factor, grade >= "A" is meaningful and returned four students.
- **Summarise:** mean total by programme is AI&DS 66.45, CSE 63.04, IT 71.25 and ECE 66.29; the programme x grade table shows no F or O grades in this sample.
- **Exchange with other components:** matrix to data frame to matrix was identical; the data frame's programme groups were applied to the matrix columns (subject means by programme); the data-frame summary became a 4 x 5 table; the list of matrices rebuilt the Task 5 array exactly.

TASK 7 - Design an Object-Oriented Student/Course Model Using R Classes (5 Marks)

Concept. S3 classes are lightweight attribute conventions: R does not check that an object obeys the class contract. S4 classes declare slots with types and an optional validity function that runs on every new() and validObject() call. Reference Classes add mutable objects; they are used in Task 11.

- **Final candidate (S4):** class Student with slots reg_no, name, programme, attendance, marks, a validity function, methods update_marks(), eligibility() and average_mark(), and a show() method.
- **Comparative prototype (S3):** student_s3 with a constructor-time check, update_marks_s3() and a print method.
- Objects are built from the Task 2 and Task 4 data (student 192324006).

R code

```
# TASK 7: Object-oriented Student model (S4 final design, S3 comparative prototype)
options(width = 85)

# ---------- S4 : final candidate ----------
setClass("Student",
  representation(reg_no = "character", name = "character", programme = "character",
                 attendance = "numeric", marks = "numeric"),
  validity = function(object) {
    errs <- character()
    if (length(object@reg_no) != 1 || !grepl("^[0-9]{9}$", object@reg_no))
      errs <- c(errs, "reg_no must be one 9-digit string")
    if (length(object@attendance) != 1 || is.na(object@attendance) ||
        object@attendance < 0 || object@attendance > 100)
      errs <- c(errs, "attendance must be one number in 0-100")
    if (!object@programme %in% c("AI&DS", "CSE", "IT", "ECE"))
      errs <- c(errs, "programme is not a valid category")
    if (length(object@marks) > 0 &&
        (is.null(names(object@marks)) || any(is.na(object@marks)) ||
         any(object@marks < 0 | object@marks > 100)))
      errs <- c(errs, "marks must be named values in 0-100")
    if (length(errs)) errs else TRUE
  })
```

```
setGeneric("update_marks", function(obj, subject, value) standardGeneric("update_marks"))
setGeneric("eligibility",  function(obj, min_attendance = 75, pass_mark = 40) standardGeneric("eligibility"))
setGeneric("average_mark", function(obj) standardGeneric("average_mark"))

setMethod("update_marks", "Student", function(obj, subject, value) {
  obj@marks[subject] <- value
  validObject(obj)
  obj
})
setMethod("average_mark", "Student", function(obj) mean(obj@marks))
setMethod("eligibility", "Student", function(obj, min_attendance = 75, pass_mark = 40) {
  reasons <- character()
  if (obj@attendance < min_attendance) reasons <- c(reasons, "attendance below minimum")
  failed <- names(obj@marks)[obj@marks < pass_mark]
  if (length(failed)) reasons <- c(reasons, paste("failed:", paste(failed, collapse = ", ")))
  list(eligible = length(reasons) == 0, reasons = reasons)
})
setMethod("show", "Student", function(object) {
  cat("<Student>", object@reg_no, "-", object@name, "(", object@programme, ")\n")
  cat("  attendance:", object@attendance, "% | average mark:", round(average_mark(object), 2), "\n")
  cat("  marks:", paste(names(object@marks), object@marks, sep = "=", collapse = ", "), "\n")
})

cat("--- S4 object built from the earlier components ---\n")
i  <- which(core$reg_no == "192324006")
s4 <- new("Student", reg_no = core$reg_no[i], name = core$name[i], programme = core$programme[i],
          attendance = core$attendance[i], marks = marks[core$reg_no[i], ])
show(s4)
cat("eligibility():\n"); str(eligibility(s4))
s4b <- update_marks(s4, "EN105", 52)
cat("After update_marks(EN105 = 52): average =", round(average_mark(s4b), 2),
    "| original untouched (copy semantics):", s4@marks["EN105"], "\n")
str(eligibility(s4b))

cat("\n--- S4 test cases (expected to be rejected) ---\n")
try_new <- function(label, expr) {
```

```
 r <- tryCatch({ force(expr); "ACCEPTED" }, error = function(e) conditionMessage(e))
 cat(sprintf("%-26s -> %s\n", label, r))
}
try_new("attendance = 120", new("Student", reg_no = "192324001", name = "A", programme = "CSE",
attendance = 120, marks = c(CS101 = 50)))
try_new("attendance as text", new("Student", reg_no = "192324001", name = "A", programme = "CSE",
attendance = "85%", marks = c(CS101 = 50)))
try_new("bad programme", new("Student", reg_no = "192324001", name = "A", programme = "MECH",
attendance = 80, marks = c(CS101 = 50)))
try_new("mark = 140 via update", update_marks(s4, "CS101", 140))
try_new("unnamed marks", new("Student", reg_no = "192324001", name = "A", programme = "CSE",
attendance = 80, marks = c(50, 60)))

# ---------- S3 : comparative prototype ----------
new_student_s3 <- function(reg_no, name, programme, attendance, marks) {
 obj <- structure(list(reg_no = reg_no, name = name, programme = programme,
                       attendance = attendance, marks = marks), class = "student_s3")
 validate_student_s3(obj)
 obj
}
validate_student_s3 <- function(obj) {
 if (!is.numeric(obj$attendance) || obj$attendance < 0 || obj$attendance > 100)
   stop("attendance must be a number in 0-100", call. = FALSE)
 invisible(obj)
}
update_marks_s3 <- function(obj, subject, value) UseMethod("update_marks_s3")
update_marks_s3.student_s3 <- function(obj, subject, value) { obj$marks[subject] <- value; obj }
print.student_s3 <- function(x, ...) cat("<student_s3>", x$reg_no, x$name, "attendance", x$attendance, "\n")

cat("\n--- S3 prototype ---\n")
s3 <- new_student_s3(core$reg_no[i], core$name[i], core$programme[i], core$attendance[i],
marks[core$reg_no[i], ])
print(s3)
r <- tryCatch(new_student_s3("192324001", "A", "CSE", 120, c(CS101 = 50)), error = function(e)
conditionMessage(e))
cat("Constructor with attendance = 120 ->", r, "\n")
```

```
rogue <- structure(list(reg_no = "x", attendance = 999, marks = c(CS101 = 500)), class = "student_s3")
cat("Bypassing the constructor with structure(): accepted, attendance =", rogue$attendance, "\n")
s3u <- update_marks_s3(s3, "CS101", 140)
cat("S3 update_marks_s3(CS101 = 140) accepted silently: mark =", s3u$marks["CS101"], "\n")

cat("\n--- Measured comparison ---\n")
n <- 2000
t_s4 <- system.time(for (k in seq_len(n)) new("Student", reg_no = "192324001", name = "A", programme =
"CSE", attendance = 80, marks = c(CS101 = 50, MA102 = 60)))[["elapsed"]]
t_s3 <- system.time(for (k in seq_len(n)) new_student_s3("192324001", "A", "CSE", 80, c(CS101 = 50, MA102
= 60)))[["elapsed"]]
comparison <- data.frame(
  criterion = c("Validation enforced on every new()", "Update re-validated automatically",
                "Can be bypassed by structure()", paste("Time to build", n, "objects (s)")),
  S4 = c("Yes", "Yes (validObject)", "No", sprintf("%.2f", t_s4)),
  S3 = c("Only if constructor used", "No", "Yes", sprintf("%.2f", t_s3)))
print(comparison, right = FALSE)
cat("DECISION: S4 for the single-student object (guaranteed validity); see Task 11 for bulk storage.\n")
```

Output (captured from running the code above)

```
--- S4 object built from the earlier components ---
<Student> 192324006 - Farhan Ali ( AI&DS )
  attendance: 69 % | average mark: 49
  marks: CS101=47, MA102=59, ST103=52, DS104=49, EN105=38
eligibility():
List of 2
 $ eligible: logi FALSE
 $ reasons : chr [1:2] "attendance below minimum" "failed: EN105"
After update_marks(EN105 = 52): average = 51.8 | original untouched (copy semantics): 38
List of 2
 $ eligible: logi FALSE
 $ reasons : chr "attendance below minimum"

--- S4 test cases (expected to be rejected) ---
attendance = 120          -> invalid class "Student" object: attendance must be one number in 0-100
```

attendance as text -> invalid class "Student" object: invalid object for slot "attendance" in class "Student": got class "character", should be or extend class "numeric"

bad programme -> invalid class "Student" object: programme is not a valid category

mark = 140 via update -> invalid class "Student" object: marks must be named values in 0-100

unnamed marks -> invalid class "Student" object: marks must be named values in 0-100

--- S3 prototype ---

<student_s3> 192324006 Farhan Ali attendance 69

Constructor with attendance = 120 -> attendance must be a number in 0-100

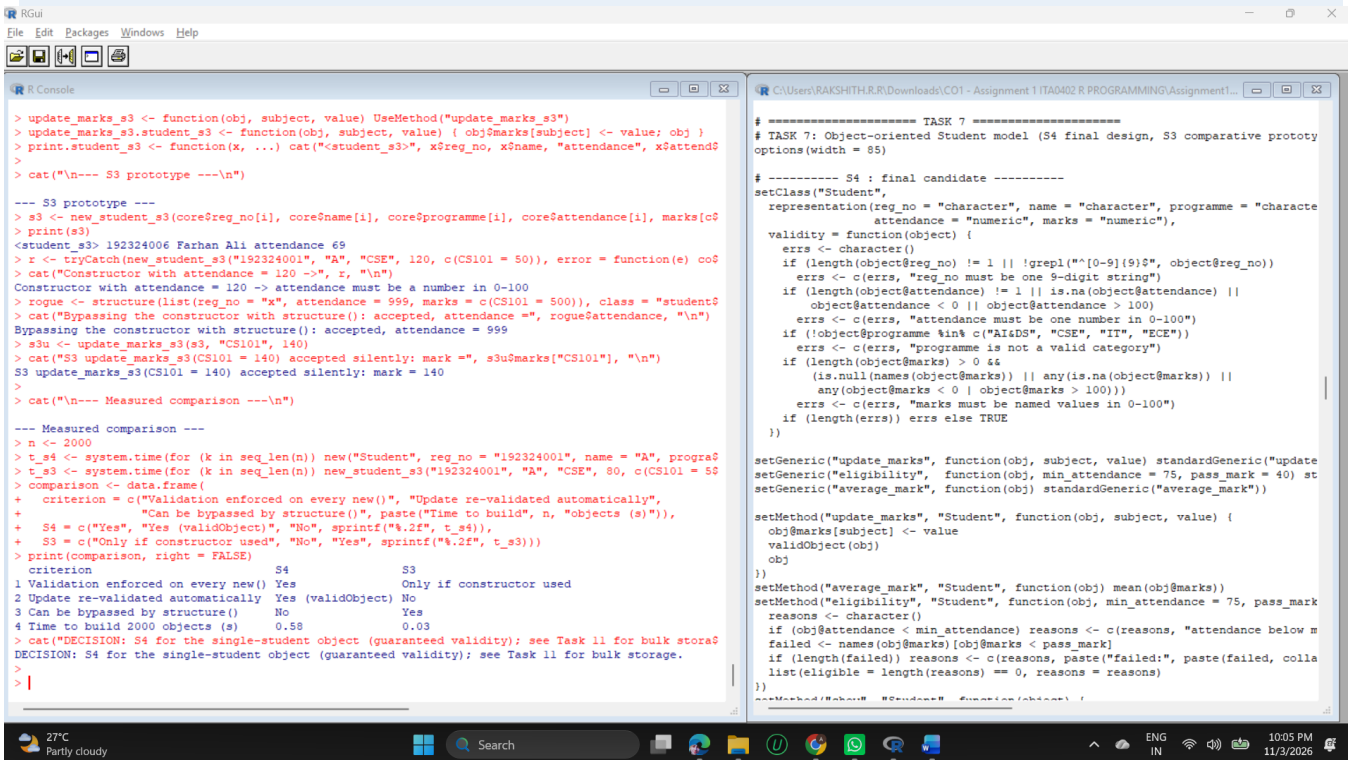
Bypassing the constructor with structure(): accepted, attendance = 999

S3 update_marks_s3(CS101 = 140) accepted silently: mark = 140

--- Measured comparison ---

	criterion	S4	S3
1	Validation enforced on every new()	Yes	Only if constructor used
2	Update re-validated automatically	Yes (validObject)	No
3	Can be bypassed by structure()	No	Yes
4	Time to build 2000 objects (s)	0.44	0.03

DECISION: S4 for the single-student object (guaranteed validity); see Task 11 for bulk storage.



Comparison evidence

Criterion	S4 (measured)	S3 (measured)
Validation on every new()	Yes: five invalid objects were rejected	Only through the constructor
Update re-validated	Yes: update_marks(s4, "CS101", 140) was rejected	No: mark 140 accepted silently
Can be bypassed	No	Yes: structure(list(attendance = 999, ...)) was accepted
Type of slots enforced	Yes: attendance "85%" rejected by the slot type	No
Time to build 2,000 objects	0.44 s	0.03 s

Justification of the final class design

- **Structure:** slots give every student the same named, typed fields, replacing positional access into loose lists.
- **Validation:** the five rejected cases (attendance 120, attendance as text, bad programme, mark 140 through update, unnamed marks) show the object cannot be created or updated into an invalid state.
- **Reuse and maintainability:** methods are generic functions (eligibility(), update_marks()), so a future Faculty or Course class can reuse the names. update_marks() returns a modified copy and leaves the original untouched (mark 38 kept while the copy shows 52), which makes changes traceable.
- **Cost:** S4 creation is roughly an order of magnitude slower than S3. For that reason S4 is used for individual validated records and at boundaries, and Task 11 shows how bulk data is held in matrices instead.

TASK 8 - Develop a Reusable R Function Library for the Framework (4 Marks)

Five functions were written, each with documented arguments, default values and an explicit return object. The comment block above each function states the contract; the test harness checks it with valid, boundary and invalid inputs.

Function	Arguments (defaults)	Return object	Integrates with
validate_marks	x, min = 0, max = 100	list: valid (logical), n_invalid, issues (data frame)	Tasks 2, 9, 10
eligibility_status	attendance, min_attendance = 75	logical vector; NA stays NA	Task 2 core
assign_grade	marks, breaks, labels	ordered factor	Task 6 data frame
summarise_marks	m, pass_mark = 40	data frame: mean, min, max, n_pass, all_pass, grade	Task 4 matrix
weighted_total	arr, weights	students x subjects numeric matrix	Task 5 array

R code

```
# TASK 8: Reusable function library
options(width = 85)

#' validate_marks(x, min = 0, max = 100)
#'   x: numeric/character vector of marks.  Returns a list: valid (logical vector),
#'   n_invalid (integer), issues (data frame: index, value, reason).
validate_marks <- function(x, min = 0, max = 100) {
 num <- suppressWarnings(as.numeric(x))
 reason <- ifelse(is.na(x), "missing",
           ifelse(is.na(num), "not numeric",
           ifelse(num < min | num > max, "out of range", NA_character_)))
 bad <- which(!is.na(reason))
 list(valid = is.na(reason), n_invalid = length(bad),
      issues = data.frame(index = bad, value = as.character(x[bad]), reason = reason[bad],
                          stringsAsFactors = FALSE))
}
```

```r
#' eligibility_status(attendance, min_attendance = 75)
#'   Returns a logical vector; NA stays NA (unknown is not "not eligible").
eligibility_status <- function(attendance, min_attendance = 75) {
 if (!is.numeric(attendance)) stop("attendance must be numeric")
 attendance >= min_attendance
}

#' assign_grade(marks, breaks, labels)
#'   Returns an ORDERED FACTOR; NA for NA; error for values outside [0, 100].
assign_grade <- function(marks, breaks = c(-Inf, 40, 55, 70, 85, Inf),
                         labels = c("F", "C", "B", "A", "O")) {
 if (any(marks < 0 | marks > 100, na.rm = TRUE)) stop("marks outside 0-100")
 cut(marks, breaks = breaks, labels = labels, right = FALSE, ordered_result = TRUE)
}

#' summarise_marks(m, pass_mark = 40)
#'   m: students x subjects numeric matrix with dimnames.
#'   Returns a data frame: reg_no, mean, min, max, n_pass, all_pass, grade.
summarise_marks <- function(m, pass_mark = 40) {
 if (is.null(rownames(m)) || is.null(colnames(m))) stop("matrix needs row and column names")
 avg <- rowMeans(m, na.rm = TRUE)
 data.frame(reg_no = rownames(m), mean = round(avg, 2),
            min = apply(m, 1, min, na.rm = TRUE), max = apply(m, 1, max, na.rm = TRUE),
            n_pass = rowSums(m >= pass_mark, na.rm = TRUE),
            all_pass = apply(m >= pass_mark, 1, all), grade = assign_grade(avg),
            row.names = NULL, stringsAsFactors = FALSE)
}

#' weighted_total(arr, weights)
#'   arr: students x subjects x components array with named 3rd dimension.
#'   weights: named numeric vector summing to 1.  Returns a students x subjects matrix.
weighted_total <- function(arr, weights) {
 if (abs(sum(weights) - 1) > 1e-8) stop("weights must sum to 1")
 if (!setequal(names(weights), dimnames(arr)[[3]])) stop("weights do not match components")
 Reduce(`+`, lapply(names(weights), function(k) arr[, , k] * weights[[k]]))
```

```
}

cat("--- Function signatures ---\n")
for (fn in c("validate_marks", "eligibility_status", "assign_grade", "summarise_marks", "weighted_total"))
  cat(fn, "(", paste(names(formals(get(fn))), collapse = ", "), ")\n", sep = "")

check <- function(id, fn, case, ok) data.frame(id = id, fn = fn, case = case,
                                               result = if (isTRUE(ok)) "PASS" else "FAIL")
err <- function(expr) tryCatch({ expr; FALSE }, error = function(e) TRUE)
tests <- rbind(
  check(1,  "validate_marks", "valid vector",       validate_marks(c(0, 55, 100))$n_invalid == 0),
  check(2,  "validate_marks", "boundary 100.01",    validate_marks(100.01)$issues$reason == "out of range"),
  check(3,  "validate_marks", "invalid text",       validate_marks("AB")$issues$reason == "not numeric"),
  check(4,  "validate_marks", "missing NA",         validate_marks(NA_real_)$issues$reason == "missing"),
  check(5,  "validate_marks", "custom range 0-50",  validate_marks(60, max = 50)$n_invalid == 1),
  check(6,  "eligibility_status", "boundary 75",    isTRUE(eligibility_status(75))),
  check(7,  "eligibility_status", "boundary 74.99", isFALSE(eligibility_status(74.99))),
  check(8,  "eligibility_status", "NA stays NA",    is.na(eligibility_status(NA_real_))),
  check(9,  "eligibility_status", "text rejected",  err(eligibility_status("85%"))),
  check(10, "assign_grade", "40 -> C (boundary)",   as.character(assign_grade(40)) == "C"),
  check(11, "assign_grade", "39.9 -> F",            as.character(assign_grade(39.9)) == "F"),
  check(12, "assign_grade", "100 -> O",             as.character(assign_grade(100)) == "O"),
  check(13, "assign_grade", "105 rejected",         err(assign_grade(105))),
  check(14, "assign_grade", "ordered factor",       is.ordered(assign_grade(c(10, 90)))),
  check(15, "summarise_marks", "matrix with names", nrow(summarise_marks(marks)) == 12),
  check(16, "summarise_marks", "no dimnames",       err(summarise_marks(unname(marks)))),
  check(17, "weighted_total", "weights sum to 1",   all(dim(weighted_total(arr, w)) == c(12, 5))),
  check(18, "weighted_total", "weights sum 1.2",    err(weighted_total(arr, c(internal = .4, model = .4, practical =
.4)))),
  check(19, "weighted_total", "wrong names",        err(weighted_total(arr, c(a = .3, b = .3, c = .4))))
)
cat("\n--- Test results (valid, boundary, invalid inputs) ---\n")
print(tests, row.names = FALSE)
cat("Passed:", sum(tests$result == "PASS"), "of", nrow(tests), "\n")

cat("\n--- Integration: summarise_marks() on the Task 4 matrix ---\n")
```

```
print(head(summarise_marks(marks), 6))
cat("\n--- Integration: weighted_total() on the Task 5 array ---\n")
cat("Equal to Task 5 'total':", isTRUE(all.equal(weighted_total(arr, w), total, check.attributes = FALSE)), "\n")
```

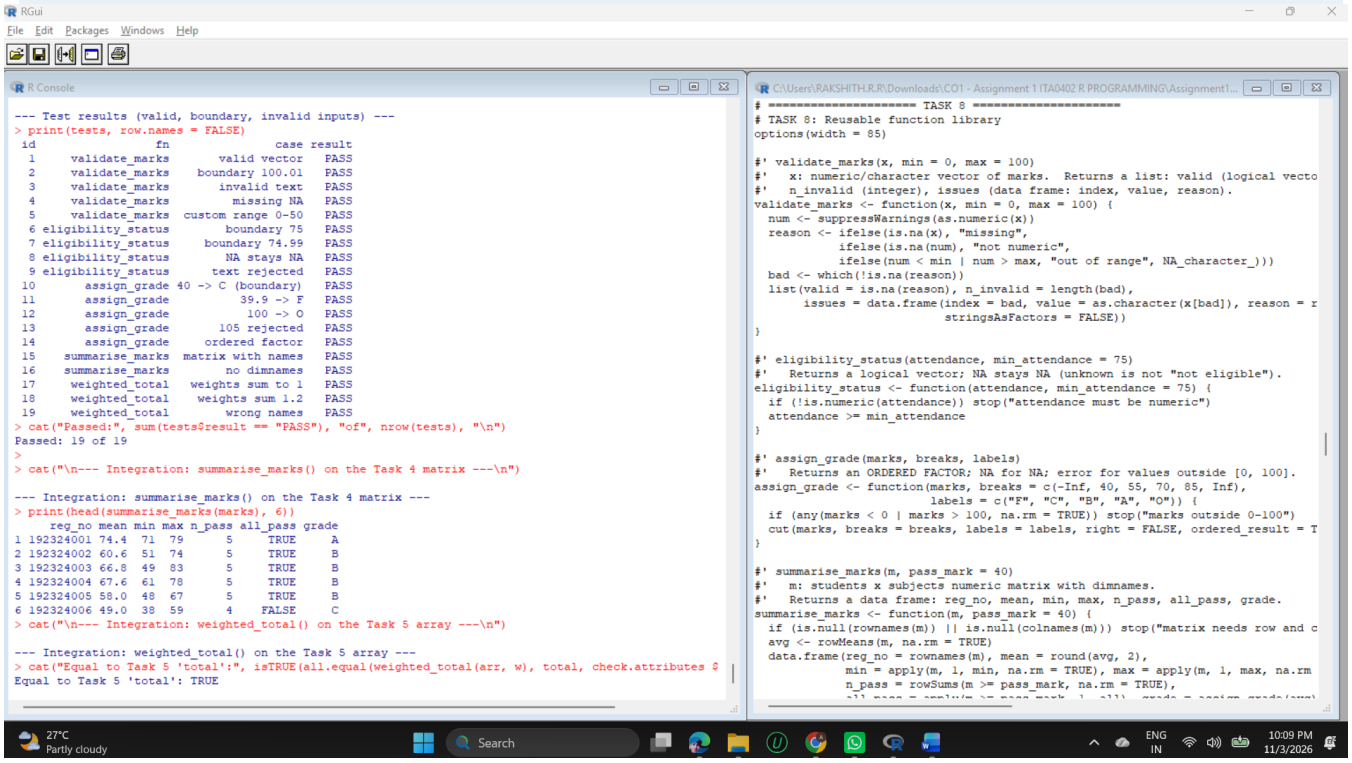
Output (captured from running the code above)

```
--- Function signatures ---
validate_marks(x, min, max)
eligibility_status(attendance, min_attendance)
assign_grade(marks, breaks, labels)
summarise_marks(m, pass_mark)
weighted_total(arr, weights)

--- Test results (valid, boundary, invalid inputs) ---
 id              fn              case result
  1   validate_marks       valid vector   PASS
  2   validate_marks    boundary 100.01   PASS
  3   validate_marks       invalid text   PASS
  4   validate_marks         missing NA   PASS
  5   validate_marks  custom range 0-50   PASS
  6 eligibility_status        boundary 75   PASS
  7 eligibility_status     boundary 74.99   PASS
  8 eligibility_status        NA stays NA   PASS
  9 eligibility_status      text rejected   PASS
 10      assign_grade 40 -> C (boundary)   PASS
 11      assign_grade          39.9 -> F   PASS
 12      assign_grade           100 -> O   PASS
 13      assign_grade       105 rejected   PASS
 14      assign_grade     ordered factor   PASS
 15  summarise_marks  matrix with names   PASS
 16  summarise_marks        no dimnames   PASS
 17   weighted_total   weights sum to 1   PASS
 18   weighted_total    weights sum 1.2   PASS
 19   weighted_total        wrong names   PASS
Passed: 19 of 19
```

```
--- Integration: summarise_marks() on the Task 4 matrix ---
    reg_no mean min max n_pass all_pass grade
1 192324001 74.4  71  79      5     TRUE     A
2 192324002 60.6  51  74      5     TRUE     B
3 192324003 66.8  49  83      5     TRUE     B
4 192324004 67.6  61  78      5     TRUE     B
5 192324005 58.0  48  67      5     TRUE     B
6 192324006 49.0  38  59      4    FALSE     C

--- Integration: weighted_total() on the Task 5 array ---
Equal to Task 5 'total': TRUE
```



Interpretation

- **Validation function:** validate_marks() separates missing, non-numeric and out-of-range values and returns the positions, so a caller can log them (used in Task 9).
- **Calculation function:** weighted_total() refuses weights that do not sum to 1 or do not match the component names, which prevents a silently wrong total.
- **Complex return objects:** summarise_marks() returns a data frame and validate_marks() returns a list containing a data frame.

- **Integration:** summarise_marks() ran directly on the Task 4 matrix, and weighted_total() reproduced the Task 5 weighted totals exactly.
- **Boundaries:** 75 attendance is eligible and 74.99 is not; mark 40 is grade C and 39.9 is F; 100 is valid and 100.01 is not.
- **Result:** 19 of 19 tests passed. The first run showed test 17 failing because of an operator-precedence mistake in the test itself; it was corrected (see the Task 9 failure record).

TASK 9 - Integrate, Test and Refine the Complete R Framework (4 Marks)

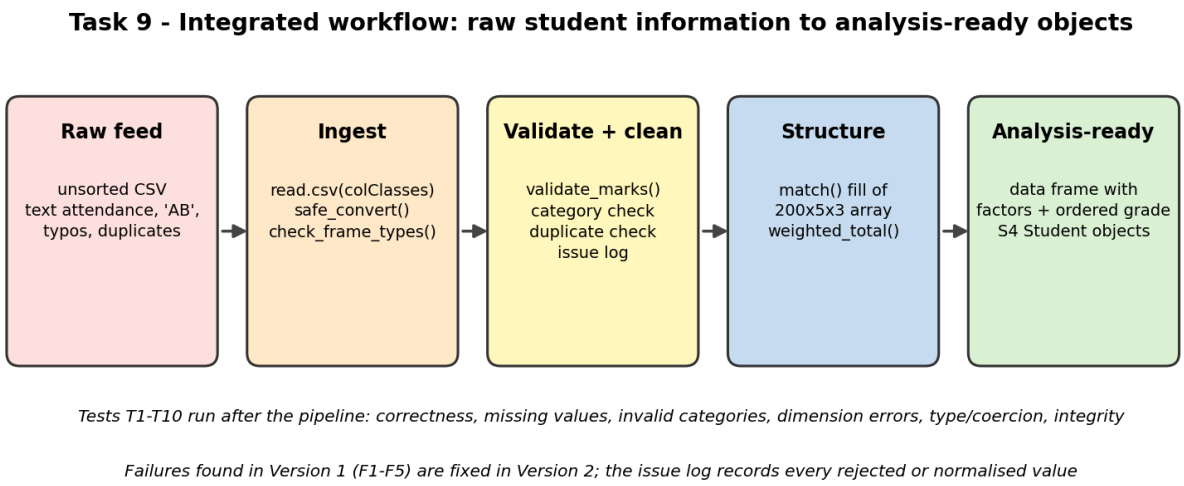


Figure 5: Integrated workflow from raw student information to analysis-ready objects

Dataset. A representative raw feed was generated: 200 students, 5 subjects, 3 components, so 3,000 mark records are expected. The feed is unsorted and read back from CSV files, as it would arrive in practice. These defects were injected on purpose: two text values in attendance ("85%", "n/a"), one impossible attendance (132), three programme errors ("cse", " IT", "MECH"), one duplicate ID, 14 absent markers ("AB"), 6 marks of 105, 10 missing marks and 8 records that never arrived (2,992 rows were read).

Method. Version 1 is the naive pipeline, written to record what goes wrong. Version 2 uses the framework components from Tasks 2, 3 and 8. Ten systematic tests then check the result.

R code

```
# TASK 9: Integrated prototype - raw feed -> validated -> structured -> analysis-ready
options(width = 85)
set.seed(909)
N <- 200
subj <- subjects$code; comp <- c("internal", "model", "practical")
ids  <- sprintf("1923%05d", 1:N)

# ---- 1. Build a messy RAW feed (as it would arrive in files) ----
raw_core <- data.frame(
  reg_no = ids, name = paste("Student", 1:N),
  programme = sample(c("AI&DS", "CSE", "IT", "ECE"), N, TRUE, prob = c(.35, .3, .2, .15)),
```

```
  attendance = as.character(round(pmin(100, rnorm(N, 84, 9)), 1)), stringsAsFactors = FALSE)
raw_core$programme[c(5, 17, 40)] <- c("cse", " IT", "MECH")      # case / space / invalid category
raw_core$attendance[c(3, 9)] <- c("85%", "n/a")                  # text in numeric field
raw_core$attendance[12] <- "132"                                 # impossible value
raw_core$reg_no[60] <- raw_core$reg_no[59]                       # duplicate ID
ability <- rnorm(N, 67, 9)
grid <- expand.grid(reg_no = ids, subject = subj, component = comp, stringsAsFactors = FALSE)
grid$mark <- round(pmin(100, pmax(0, ability[match(grid$reg_no, ids)] + rnorm(nrow(grid), 2, 9))))
grid <- grid[sample(nrow(grid)), ]                               # feed arrives unsorted
grid$mark <- as.character(grid$mark)
grid$mark[sample(nrow(grid), 14)] <- "AB"                        # absent marker
grid$mark[sample(nrow(grid), 6)]  <- "105"                       # out of range
grid$mark[sample(nrow(grid), 10)] <- NA                          # missing
grid <- grid[-sample(nrow(grid), 8), ]                           # 8 records never arrived
tf <- tempfile(fileext = ".csv"); write.csv(grid, tf, row.names = FALSE)
tc <- tempfile(fileext = ".csv"); write.csv(raw_core, tc, row.names = FALSE)
in_core  <- read.csv(tc, stringsAsFactors = FALSE, colClasses = c(reg_no = "character"))
in_marks <- read.csv(tf, stringsAsFactors = FALSE, colClasses = c(reg_no = "character"))
cat("Raw core :", nrow(in_core), "rows | attendance read as", class(in_core$attendance), "\n")
cat("Raw marks:", nrow(in_marks), "rows (expected", N * 5 * 3, ") | mark read as", class(in_marks$mark), "\n")

# ---- 2. VERSION 1 (naive pipeline): record the failures ----
cat("\n=========== VERSION 1 : naive pipeline ===========\n")
v1_att  <- suppressWarnings(as.numeric(in_core$attendance))
cat("F1  attendance as.numeric(): NA introduced silently =", sum(is.na(v1_att)), "(expected 1 'n/a' + 1 '85%')\n")
cat("F2  attendance 132 accepted, max =", max(v1_att, na.rm = TRUE), "\n")
v1_prog <- factor(in_core$programme)
cat("F3  factor(programme) created", nlevels(v1_prog), "levels instead of 4:", paste(levels(v1_prog), collapse =
", "), "\n")
v1_arr <- array(suppressWarnings(as.numeric(in_marks$mark)), dim = c(N, 5, 3))
true_cell <- in_marks$mark[in_marks$reg_no == ids[1] & in_marks$subject == "CS101" &
in_marks$component == "internal"]
cat("F4  positional fill: cell [1,1,1] holds", v1_arr[1, 1, 1], "but student 1 CS101 internal is", true_cell, "\n")
cat("F5  duplicated IDs undetected:", sum(duplicated(in_core$reg_no)), "duplicate\n")

# ---- 3. VERSION 2 (refined pipeline using Tasks 2, 3 and 8) ----
```

```
cat("\n========== VERSION 2 : refined pipeline ==========\n")
issues <- list()
log_issue <- function(src, id, field, problem)
  issues[[length(issues) + 1]] <<- data.frame(source = src, id = id, field = field, problem = problem)

clean_core <- in_core
prog_raw <- clean_core$programme
clean_core$programme <- toupper(trimws(clean_core$programme))
for (k in which(prog_raw != clean_core$programme & clean_core$programme %in% valid_programmes))
  log_issue("core", clean_core$reg_no[k], "programme", "case/space fixed")
pct <- grepl("%$", clean_core$attendance)
for (k in which(pct)) log_issue("core", clean_core$reg_no[k], "attendance", "% removed")
att <- safe_convert(sub("%$", "", clean_core$attendance), "numeric")
for (k in att$failed_index) log_issue("core", clean_core$reg_no[k], "attendance", "non-numeric")
va <- validate_marks(att$value)
for (k in va$issues$index[va$issues$reason == "out of range"]) log_issue("core", clean_core$reg_no[k],
"attendance", "out of range")
clean_core$attendance <- ifelse(va$valid, att$value, NA)
bad_prog <- which(!clean_core$programme %in% valid_programmes)
for (k in bad_prog) log_issue("core", clean_core$reg_no[k], "programme", "bad category")
clean_core$programme[bad_prog] <- NA
for (k in which(duplicated(clean_core$reg_no))) log_issue("core", clean_core$reg_no[k], "reg_no", "duplicate")
clean_core <- clean_core[!duplicated(clean_core$reg_no), ]
clean_core$eligible <- eligibility_status(clean_core$attendance)

mk <- safe_convert(in_marks$mark, "numeric")
vm <- validate_marks(mk$value)
for (k in mk$failed_index) log_issue("marks", in_marks$reg_no[k], paste(in_marks$subject[k],
in_marks$component[k]), "non-numeric")
for (k in vm$issues$index[vm$issues$reason == "out of range"]) log_issue("marks", in_marks$reg_no[k],
paste(in_marks$subject[k], in_marks$component[k]), "out of range")
in_marks$mark <- ifelse(vm$valid, mk$value, NA)

arr2 <- array(NA_real_, dim = c(length(ids), 5, 3),
              dimnames = list(student = ids, subject = subj, component = comp))
```

```
arr2[cbind(match(in_marks$reg_no, ids), match(in_marks$subject, subj), match(in_marks$component, comp))]
<- in_marks$mark
n_cells <- length(arr2); n_na <- sum(is.na(arr2))
cat("Array", paste(dim(arr2), collapse = "x"), "built by NAME matching; NA cells =", n_na,
    "(", sum(!vm$valid), "flagged +", n_cells - nrow(in_marks), "never arrived )\n")

w9 <- c(internal = 0.3, model = 0.3, practical = 0.4)
tot2 <- weighted_total(arr2, w9)
avg2 <- rowMeans(tot2, na.rm = TRUE)
analysis2 <- data.frame(reg_no = clean_core$reg_no, programme = factor(clean_core$programme, levels =
valid_programmes),
                        attendance = clean_core$attendance, eligible = clean_core$eligible,
                        avg_total = round(avg2[clean_core$reg_no], 2),
                        grade = assign_grade(avg2[clean_core$reg_no]), stringsAsFactors = FALSE)
cat("Analysis-ready frame:", nrow(analysis2), "rows\n"); str(analysis2)
cat("\nIssue log (", length(issues), "entries ); counts by problem:\n")
issue_df <- do.call(rbind, issues); print(table(issue_df$source, issue_df$problem))
cat("\nSample Student S4 object from the pipeline:\n")
k <- analysis2$reg_no[1]
show(new("Student", reg_no = k, name = "Student 1", programme = as.character(analysis2$programme[1]),
         attendance = analysis2$attendance[1], marks = round(tot2[k, ], 1)))

# ---- 4. Systematic tests ----
cat("\n========== Systematic test cases ==========\n")
T <- function(id, area, desc, ok) data.frame(id = id, area = area, test = desc, result = if (isTRUE(ok)) "PASS" else
"FAIL")
res <- rbind(
  T("T1", "correctness", "student 1 CS101 total recomputed by hand",
    isTRUE(all.equal(tot2[1, "CS101"], sum(arr2[1, "CS101", ] * w9)))),
  T("T2", "correctness", "array cell equals raw record (by name)",
    { r <- in_marks[10, ]; isTRUE(all.equal(arr2[r$reg_no, r$subject, r$component], r$mark)) }),
  T("T3", "missing values", "AB/missing/dropped become NA, not 0",
    sum(is.na(arr2)) == (sum(!vm$valid) + n_cells - nrow(in_marks)) && !any(arr2 == 0, na.rm = TRUE)),
  T("T4", "invalid category", "invalid programmes flagged -> NA", sum(is.na(analysis2$programme)) ==
length(bad_prog)),
  T("T5", "dimension error", "3x3 matrix into 200x5 slice fails",
```

```
  tryCatch({ arr2[, , 1] <- matrix(0, 3, 3); FALSE }, error = function(e) TRUE)),
 T("T6", "dimension error", "unknown student ID gives subscript error",
  tryCatch({ arr2["NOPE", 1, 1]; FALSE }, error = function(e) TRUE)),
 T("T7", "type / coercion", "text attendance: 1 unusable + 1 normalised",
  length(att$failed_index) == 1 && sum(pct) == 1),
 T("T8", "type / coercion", "no mark above 100 remains", max(arr2, na.rm = TRUE) <= 100),
 T("T9", "integrity", "duplicate ID detected and removed", !any(duplicated(analysis2$reg_no)) &&
sum(duplicated(in_core$reg_no)) == 1),
 T("T10", "integrity", "array -> long -> array round trip",
  { lg <- as.data.frame.table(arr2, responseName = "mark", stringsAsFactors = FALSE)
    bk <- xtabs(mark ~ student + subject + component, data = lg, addNA = FALSE)
    isTRUE(all.equal(unclass(bk)[ids, subj, comp][!is.na(arr2)], arr2[!is.na(arr2)], check.attributes = FALSE)) })
)
print(res, row.names = FALSE); cat("Passed:", sum(res$result == "PASS"), "of", nrow(res), "\n")

# ---- 5. Failure / refinement record ----
cat("\n========== Failure and refinement record ==========\n")
fr <- data.frame(
 id = paste0("F", 1:8),
 failure_observed = c("'n/a' became NA with no record", "attendance 132 accepted",
  "typos created 7 factor levels (not 4)", "positional fill misplaced records",
  "duplicate reg_no passed through", "Task 8 test 17 failed (test bug)",
  "identical() FALSE on equal matrices", "round() failed on factor column"),
 refinement = c("safe_convert() logs index + value", "range check on attendance",
  "toupper/trimws + match valid list", "fill by match() on IDs/names",
  "duplicated() check + log", "all(dim(..) == c(12, 5))",
  "all.equal() + explicit ID check", "round numeric columns only"))
print(fr, right = FALSE)
```

Output (captured from running the code above)

```
Raw core : 200 rows | attendance read as character
Raw marks: 2992 rows (expected 3000 ) | mark read as character

========== VERSION 1 : naive pipeline ==========
F1  attendance as.numeric(): NA introduced silently = 2 (expected 1 'n/a' + 1 '85%')
```

```
F2  attendance 132 accepted, max = 132
F3  factor(programme) created 7 levels instead of 4:  IT, AI&DS, CSE, ECE, IT, MECH, cse
F4  positional fill: cell [1,1,1] holds 65 but student 1 CS101 internal is 59
F5  duplicated IDs undetected: 1 duplicate

========== VERSION 2 : refined pipeline ==========
Array 200x5x3 built by NAME matching; NA cells = 38 ( 30 flagged + 8 never arrived )
Analysis-ready frame: 199 rows
'data.frame':    199 obs. of  6 variables:
 $ reg_no    : chr  "192300001" "192300002" "192300003" "192300004" ...
 $ programme : Factor w/ 4 levels "AI&DS","CSE",..: 1 4 1 1 2 2 2 2 1 ...
 $ attendance: num  84.9 77.1 85 75 85.3 70.2 90.2 90.6 NA 83.6 ...
 $ eligible  : logi  TRUE TRUE TRUE TRUE TRUE FALSE ...
 $ avg_total : num  65.7 59.6 73.5 65.6 78.8 ...
 $ grade     : Ord.factor w/ 5 levels "F"<"C"<"B"<"A"<..: 3 3 4 3 4 3 4 4 4 4 ...

Issue log ( 27 entries ); counts by problem:

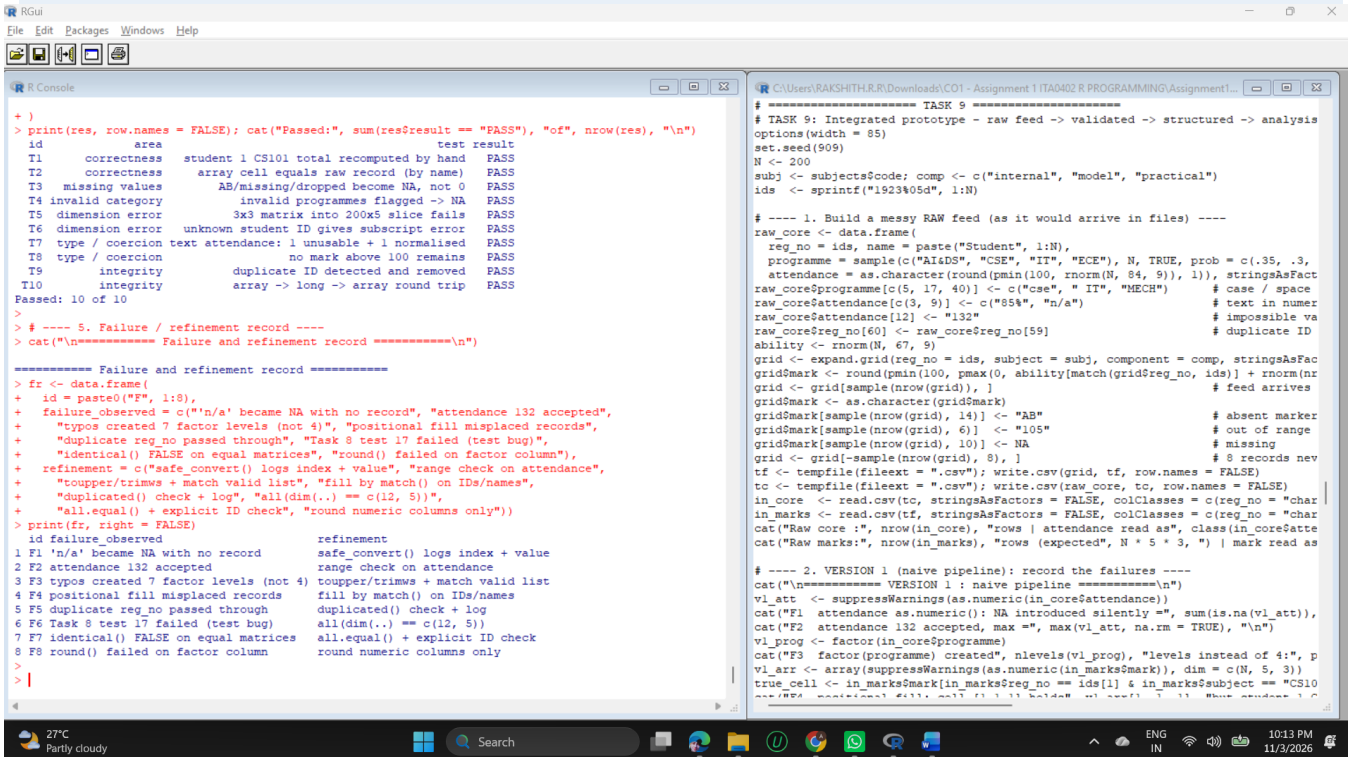
       % removed bad category case/space fixed duplicate non-numeric out of range
  core         1          1              2        1         1          1
  marks        0          0              0        0        14          6

Sample Student S4 object from the pipeline:
<Student> 192300001 - Student 1 ( AI&DS )
  attendance: 84.9 % | average mark: 65.7
  marks: CS101=64.3, MA102=60.8, ST103=64, DS104=73.1, EN105=66.3

========== Systematic test cases ==========
  id            area                                test result
  T1      correctness   student 1 CS101 total recomputed by hand   PASS
  T2      correctness     array cell equals raw record (by name)   PASS
  T3   missing values        AB/missing/dropped become NA, not 0   PASS
  T4 invalid category           invalid programmes flagged -> NA   PASS
  T5  dimension error          3x3 matrix into 200x5 slice fails   PASS
  T6  dimension error   unknown student ID gives subscript error   PASS
  T7  type / coercion text attendance: 1 unusable + 1 normalised   PASS
```

```
 T8  type / coercion              no mark above 100 remains   PASS
 T9       integrity         duplicate ID detected and removed   PASS
T10       integrity          array -> long -> array round trip   PASS
Passed: 10 of 10


========== Failure and refinement record ==========
 id failure_observed                    refinement
1 F1 'n/a' became NA with no record       safe_convert() logs index + value
2 F2 attendance 132 accepted              range check on attendance
3 F3 typos created 7 factor levels (not 4) toupper/trimws + match valid list
4 F4 positional fill misplaced records     fill by match() on IDs/names
5 F5 duplicate reg_no passed through       duplicated() check + log
6 F6 Task 8 test 17 failed (test bug)      all(dim(..) == c(12, 5))
7 F7 identical() FALSE on equal matrices   all.equal() + explicit ID check
8 F8 round() failed on factor column       round numeric columns only
```



Failures recorded in Version 1 and the refinements

ID	Failure observed	Refinement in Version 2	Type

F1	as.numeric() turned "n/a" into NA with no record (2 NA values appeared silently)	safe_convert() reports failing position and value; the % sign is removed deliberately and logged	Data pipeline
F2	Attendance 132 accepted (maximum 132)	Range check with validate_marks(); value logged and set to NA	Data pipeline
F3	Typos created 7 factor levels instead of 4	toupper(trimws()), match against the valid list, invalid becomes NA and is logged	Data pipeline
F4	Positional array fill: cell [1,1,1] held 65 but the true mark was 59	Fill by match() on IDs, subjects and components	Data pipeline
F5	Duplicate registration number passed through	duplicated() check, first kept, duplicate logged	Data pipeline
F6	Task 8 test 17 failed	Test rewritten as all(dim(...) == c(12, 5)) (precedence error in the test)	Developme nt
F7	identical() returned FALSE for equal matrices	Named dimnames differ, so all.equal() plus an explicit ID check is used	Developme nt
F8	round() stopped on a data frame with a factor column	Round only the numeric columns	Developme nt

Interpretation

- **Missing values:** the Version 2 array has 38 NA cells = 30 flagged marks + 8 records that never arrived. None were converted to 0, so means are not biased downwards.
- **Issue log:** 27 entries - 14 non-numeric marks, 6 out-of-range marks, and in the core data one duplicate, one bad category, one impossible attendance, one non-numeric attendance, one removed % sign and two case/space corrections.
- **Tests:** all 10 passed (correctness, missing values, invalid category, two dimension errors, type and coercion, out-of-range, duplicate and round trip).
- **Outcome:** 199 clean student rows (one duplicate removed) with factors, an ordered grade and a working S4 Student object.

TASK 10 - Create an Evidence-Based Scalability and Maintainability Evaluation (4 Marks)

Setup. Datasets of 5,000 and 100,000 students were built with 25 subjects and 3 components (375,000 and 7,500,000 array cells). Four operations were measured: (A) weighted total, (B) validation of every mark, (C) group mean by programme and (D) conversion of the array to a long data frame. Operations A, B and C were each implemented in a first version and a vectorised version. Time is elapsed seconds, median of 3 runs at 5,000 students and a single run at 100,000.

R code

```
# TASK 10: Evidence-based scalability evaluation
options(width = 85)
set.seed(10)
w10 <- c(internal = 0.3, model = 0.3, practical = 0.4)
build_data <- function(n, n_subj = 25) {
  ids <- sprintf("S%06d", seq_len(n)); sj <- sprintf("SUB%02d", seq_len(n_subj))
  a <- array(round(runif(n * n_subj * 3, 30, 100)), dim = c(n, n_subj, 3),
             dimnames = list(student = ids, subject = sj, component = names(w10)))
  d <- data.frame(reg_no = ids, programme = factor(sample(valid_programmes, n, TRUE), levels = valid_programmes),
                  stringsAsFactors = FALSE)
  list(arr = a, df = d)
}
timeit <- function(expr, reps = 1) {
  e <- substitute(expr); env <- parent.frame()
  median(sapply(seq_len(reps), function(i) system.time(eval(e, env))[["elapsed"]]))
}
rows <- list()
for (n in c(5000, 100000)) {
  bd <- build_data(n); a <- bd$arr; d <- bd$df
  reps <- if (n == 5000) 3 else 1
  # Operation A: weighted total (v1 apply, v2 vectorised)
  tA1 <- timeit(apply(a, c(1, 2), function(x) sum(x * w10)), reps)
  tA2 <- timeit(weighted_total(a, w10), reps)
  tot <- weighted_total(a, w10); d$avg <- rowMeans(tot)
  # Operation B: validation of every mark (v1 per-student loop, v2 vectorised)
```

```
 tB1 <- timeit(vapply(seq_len(n), function(i) validate_marks(a[i, , ])$n_invalid, numeric(1)), reps)
 tB2 <- timeit(sum(a < 0 | a > 100 | is.na(a)), reps)
 # Operation C: group summary by programme (v1 aggregate, v2 tapply)
 tC1 <- timeit(aggregate(avg ~ programme, data = d, FUN = mean), reps)
 tC2 <- timeit(tapply(d$avg, d$programme, mean), reps)
 # Operation D: array -> long data frame
 tD <- timeit(as.data.frame.table(a, responseName = "mark", stringsAsFactors = FALSE), 1)
 rows[[length(rows) + 1]] <- data.frame(
   students = n, subjects = 25, components = 3, cells = prod(dim(a)),
   array_MB = round(as.numeric(object.size(a)) / 1024^2, 1),
   A_apply = tA1, A_vector = tA2, B_loop = tB1, B_vector = tB2,
   C_aggregate = tC1, C_tapply = tC2, D_to_long = tD)
 rm(bd, d, a, tot); invisible(gc())
}
res <- do.call(rbind, rows)
cat("--- Experimental setup ---\n")
cat("Students: 5,000 and 100,000 | subjects: 25 | components: 3 | R", R.version.string, "\n")
cat("Timing = elapsed seconds (median of 3 runs at 5,000; single run at 100,000)\n\n")
show_res <- res[, 1:5]; show_res$students <- format(show_res$students, big.mark = ",", scientific = FALSE)
show_res$cells <- format(show_res$cells, big.mark = ",", scientific = FALSE)
print(show_res, row.names = FALSE)
cmp <- data.frame(
 operation = c("A weighted total", "A weighted total", "B validate every mark", "B validate every mark",
               "C group mean by programme", "C group mean by programme", "D array -> long frame"),
 version = c("v1 apply()", "v2 vectorised", "v1 per-student loop", "v2 vectorised", "v1 aggregate()", "v2
tapply()", "as.data.frame.table"),
 s_5000 = c(res$A_apply[1], res$A_vector[1], res$B_loop[1], res$B_vector[1], res$C_aggregate[1],
res$C_tapply[1], res$D_to_long[1]),
 s_100000 = c(res$A_apply[2], res$A_vector[2], res$B_loop[2], res$B_vector[2], res$C_aggregate[2],
res$C_tapply[2], res$D_to_long[2]))
cmp$s_5000 <- round(cmp$s_5000, 3); cmp$s_100000 <- round(cmp$s_100000, 3)
cmp$growth_x <- round(cmp$s_100000 / pmax(cmp$s_5000, 0.001), 1)
cat("\n--- Comparison table (seconds) ---\n"); print(cmp, row.names = FALSE)
sp <- data.frame(operation = c("A", "B", "C"),
                 speedup_5000 = round(c(res$A_apply[1] / res$A_vector[1], res$B_loop[1] / max(res$B_vector[1],
0.001), res$C_aggregate[1] / max(res$C_tapply[1], 0.001)), 1),
```

```
            speedup_100000 = round(c(res$A_apply[2] / res$A_vector[2], res$B_loop[2] / max(res$B_vector[2],
0.001), res$C_aggregate[2] / max(res$C_tapply[2], 0.001)), 1))
cat("\n--- Speed-up of v2 over v1 (v1 time / v2 time) ---\n"); print(sp, row.names = FALSE)

png("fig/fig_t10_scalability.png", width = 1700, height = 1000, res = 170)
par(mar = c(8, 7, 4, 1))
vals <- rbind(pmax(cmp$s_5000, 0.001), pmax(cmp$s_100000, 0.001))
bp <- barplot(vals, beside = TRUE, log = "y", col = c("#9ecae1", "#3182bd"), ylim = c(0.0005, max(vals) * 3),
        names.arg = paste(cmp$operation, cmp$version, sep = "\n"), las = 2, cex.names = 0.6,
        ylab = "", main = "Operation time: 5,000 vs 100,000 students")
mtext("Elapsed seconds (log scale)", side = 2, line = 5)
legend("topleft", legend = c("5,000 students", "100,000 students"), fill = c("#9ecae1", "#3182bd"), bty = "n")
dev.off()
scal <- res
```

Output (captured from running the code above)

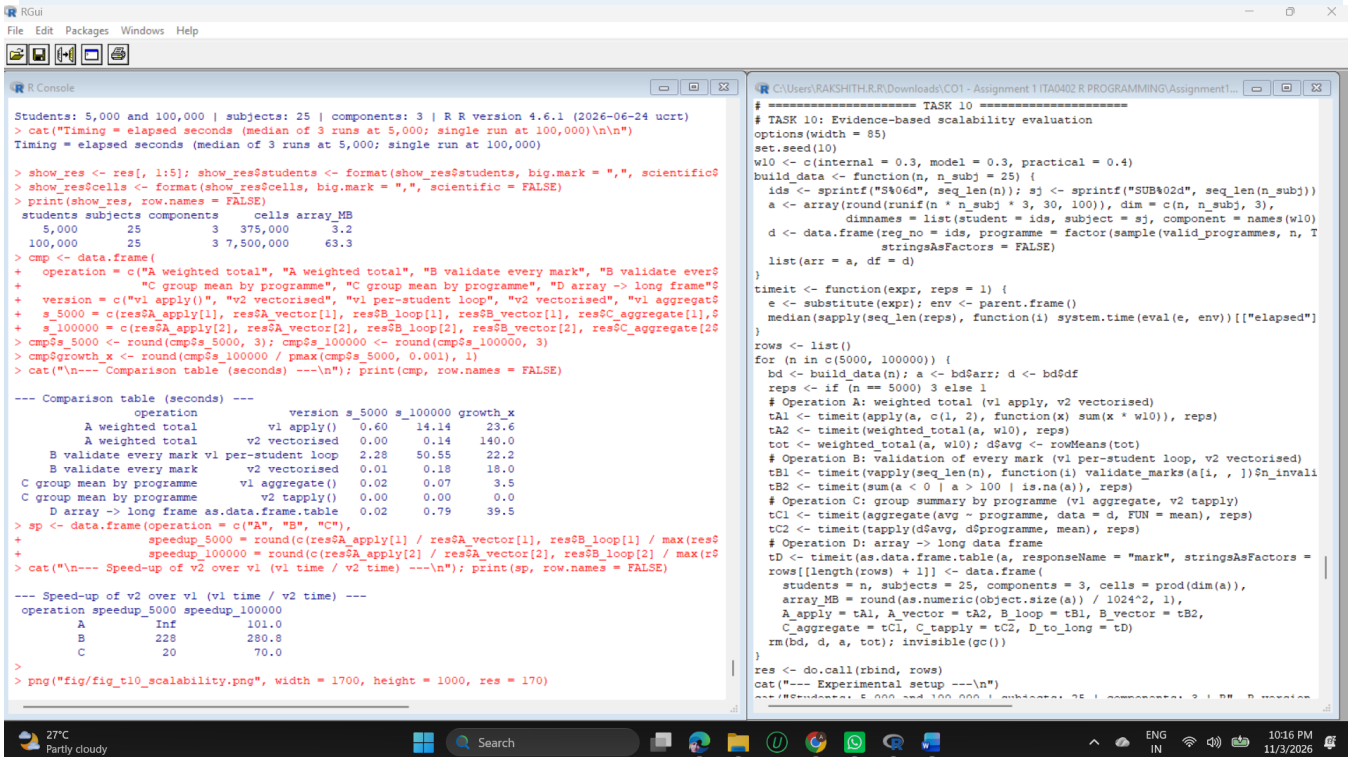
```
--- Experimental setup ---
Students: 5,000 and 100,000 | subjects: 25 | components: 3 | R R version 4.3.3 (2024-02-29)
Timing = elapsed seconds (median of 3 runs at 5,000; single run at 100,000)

 students subjects components     cells array_MB
    5,000       25          3   375,000      3.2
  100,000       25          3 7,500,000     63.3

--- Comparison table (seconds) ---
                operation            version s_5000 s_100000 growth_x
          A weighted total         v1 apply()  0.457   11.645     25.5
          A weighted total      v2 vectorised  0.004    0.083     20.8
     B validate every mark v1 per-student loop  1.530   33.244     21.7
     B validate every mark      v2 vectorised  0.004    0.137     34.2
 C group mean by programme     v1 aggregate()  0.002    0.059     29.5
 C group mean by programme       v2 tapply()  0.000    0.004      4.0
     D array -> long frame as.data.frame.table  0.023    0.753     32.7

--- Speed-up of v2 over v1 (v1 time / v2 time) ---
```

operation	speedup_5000	speedup_100000
A	114.2	140.3
B	382.5	242.7
C	2.0	14.8



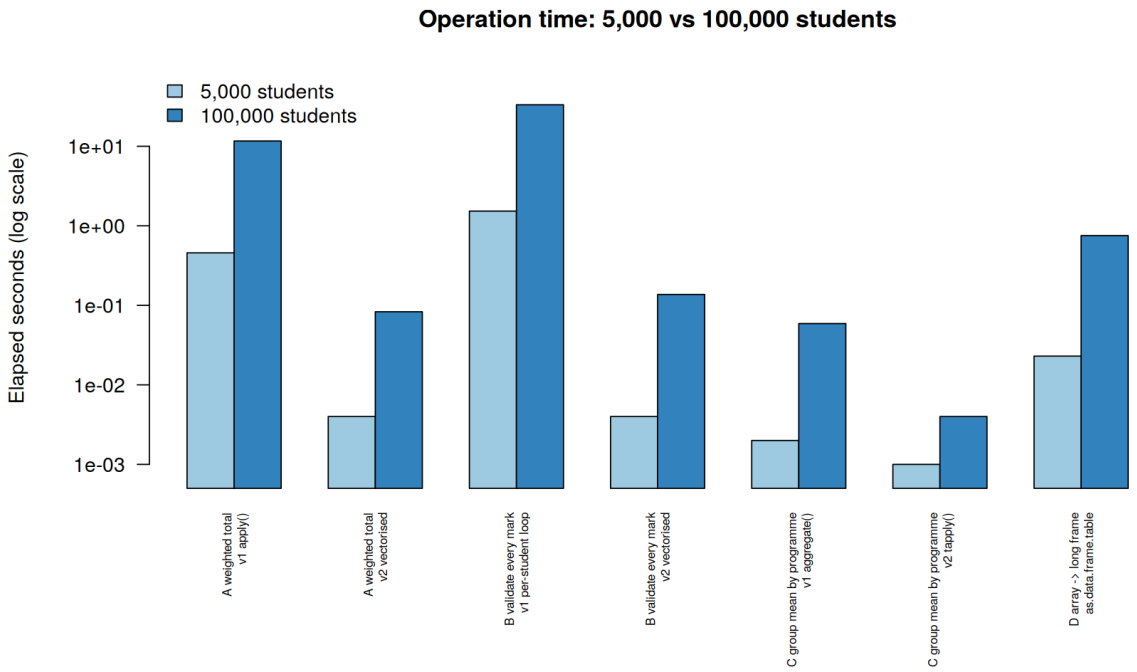


Figure 6: Operation time at 5,000 and 100,000 students (log scale)

Comparison table (seconds)

Operation	Version	5,000 students	100,000 students	Growth
A weighted total	v1 apply()	0.457	11.645	25.5x
A weighted total	v2 vectorised	0.004	0.083	20.8x
B validate every mark	v1 per-student loop	1.530	33.244	21.7x
B validate every mark	v2 vectorised	0.004	0.137	34.2x
C group mean	v1 aggregate()	0.002	0.059	29.5x
C group mean	v2 tapply()	below 0.001	0.004	-
D array to long frame	as.data.frame.table	0.023	0.753	32.7x

Object dimensions: the 100,000-student array occupies 63.3 MB (3.2 MB at 5,000). Times in the table are copied from the output above.

Interpretation and design refinement

- **Growth:** a 20-fold increase in students raised most times by 20 to 35 times, so cost grows roughly in line with data volume. Nothing grew explosively, which supports the array and matrix design as such.
- **The weak points were implementation choices, not the structures.** apply() with an R function per cell needed 11.6 s at 100,000 students and the per-student validation loop needed 33.2 s. The vectorised versions needed 0.083 s and 0.137 s, about 140 and 240 times faster.
- **Group summaries:** tapply() was about 15 times faster than aggregate() at 100,000 rows.
- **Array to long frame:** 0.75 s, producing 7.5 million rows. It is acceptable for a one-off report but too heavy for routine use, so conversion is limited to the students or components needed (chunks).
- **Refinement carried into the framework:** vectorised weighted_total() and range checks replace per-cell and per-student loops; tapply() is used for group means; long conversion is done in chunks. All six design criteria remain satisfied.
- **Limits of this evidence:** one run at 100,000 students, one machine, R 4.3.3; the 5,000-student tapply() time is below the 1 ms timer resolution.

TASK 11 - Defend the Final Architecture Under a Changed Requirement (4 Marks)

Changed requirement: 100,000 students, 40 subjects, continuous assessment (three CA components plus model and practical, and a fourth CA added later) and frequent individual-record updates, while staying easy to validate, transform and extend.

11.1 Revised architecture: retain, modify, replace, redesign

Component	Decision	Reason
Typed core vectors and validation	**Retain**	Cheap, vectorised and still correct at 100,000 rows
safe_convert(), strict_combine()	**Retain**	Ingestion rules did not change
S4 Student class	**Modify** (boundaries only)	Strong validity, but creating 100,000 objects would take about 22 s (extrapolated from Task 7)
3-D assessment array	**Redesign** (matrices held in an environment)	Adding a component rebuilds the whole array; one matrix per component can be added alone
Update by copying the structure	**Replace** (Reference Class store)	Copying a 3-D array for each update took about 160 ms
Function library	**Modify**	weighted_total() generalised to per-component weights

11.2 New component and experiments

AssessmentStore is a Reference Class holding one 100,000 x 40 matrix per component in an environment. Its update_mark() validates the value, student, subject and component, edits the cell, and appends an audit entry. A first version stored the matrices in a list field and edited them with <<-. Measurement showed that this copied a 32 MB matrix on every update, so the final version detaches the matrix from the environment, edits it while only one reference exists, and re-attaches it.

R code

```
# TASK 11: Changed requirement - 100,000 students, 40 subjects, continuous assessment
options(width = 85)
set.seed(11)
N11 <- 100000; M11 <- 40
```

```
ids11  <- sprintf("S%06d", seq_len(N11)); sub11 <- sprintf("SUB%02d", seq_len(M11))
comp11 <- c("CA1", "CA2", "CA3", "model", "practical")
w11    <- c(CA1 = 0.10, CA2 = 0.10, CA3 = 0.10, model = 0.40, practical = 0.30)

cat("--- Decision table: retain / modify / replace / redesign ---\n")
decision <- data.frame(
  component = c("Typed core vectors + validation", "safe_convert / strict_combine", "S4 Student class",
                "3-D assessment array", "Immutable update by copying", "Function library (Task 8)"),
  action = c("RETAIN", "RETAIN", "MODIFY (boundary use only)", "REDESIGN (matrices in env)",
             "REPLACE (Reference Class)", "MODIFY (per-component weights)"))
print(decision, right = FALSE)

# ---- New component: Reference Class AssessmentStore ----
# First attempt (v1): matrices kept in a list field and edited with <<-
StoreV1 <- setRefClass("StoreV1", fields = list(marks = "list"),
  methods = list(update = function(component, i, j, value) { marks[[component]][i, j] <<- value }))

# Final design (v2): matrices live in an environment; an update detaches the matrix,
# edits it while only one reference exists (no copy), and re-attaches it.
AssessmentStore <- setRefClass("AssessmentStore",
  fields = list(mats = "environment", component_names = "character", weights = "numeric",
                student_ids = "character", subject_ids = "character",
                row_index = "environment", audit = "list"),
  methods = list(
    initialize = function(student_ids = character(), subject_ids = character(),
                          components = character(), weights = numeric(), ...) {
      callSuper(student_ids = student_ids, subject_ids = subject_ids,
                component_names = components, weights = weights, ...)
      if (length(components) > 0) {
        if (abs(sum(weights) - 1) > 1e-8) stop("weights must sum to 1")
        mats <<- new.env()
        for (k in components)
          assign(k, matrix(NA_real_, length(student_ids), length(subject_ids),
                           dimnames = list(student_ids, subject_ids)), envir = mats)
        row_index <<- list2env(as.list(setNames(seq_along(student_ids), student_ids)), hash = TRUE)
      }
```

```
  },
  get_matrix = function(component) get(component, envir = mats),
  update_mark = function(reg_no, subject, component, value) {
    if (!is.numeric(value) || length(value) != 1 || is.na(value) || value < 0 || value > 100)
      stop("value must be one number in 0-100")
    i <- row_index[[reg_no]]; j <- match(subject, subject_ids)
    if (is.null(i)) stop("unknown student: ", reg_no)
    if (is.na(j)) stop("unknown subject: ", subject)
    if (!component %in% component_names) stop("unknown component: ", component)
    m <- get(component, envir = mats); rm(list = component, envir = mats)
    old <- m[i, j]; m[i, j] <- value
    assign(component, m, envir = mats)
    audit[[length(audit) + 1]] <<- list(id = reg_no, subject = subject, component = component,
                                        old = old, new = value)
    invisible(.self)
  },
  get_mark = function(reg_no, subject, component) get(component, envir = mats)[row_index[[reg_no]],
subject],
  add_component = function(name, mat, new_weights) {
    if (!identical(dimnames(mat), list(student_ids, subject_ids))) stop("matrix dimnames do not match the store")
    if (abs(sum(new_weights) - 1) > 1e-8 || !setequal(names(new_weights), c(component_names, name)))
      stop("new_weights must cover all components and sum to 1")
    assign(name, mat, envir = mats)
    component_names <<- c(component_names, name); weights <<- new_weights; invisible(.self)
  },
  weighted_total = function() {
    Reduce(`+`, lapply(names(weights), function(k) get(k, envir = mats) * weights[[k]]))
  },
  to_long = function(ids = student_ids) {
    do.call(rbind, lapply(component_names, function(k)
      data.frame(student = rep(ids, times = length(subject_ids)), subject = rep(subject_ids, each = length(ids)),
                 component = k, mark = as.vector(get(k, envir = mats)[ids, ]), stringsAsFactors = FALSE)))
  }
 ))

cat("\n--- Build the store at the changed scale ---\n")
```

```
t_build <- system.time({
  store <- AssessmentStore$new(ids11, sub11, comp11, w11)
  for (k in comp11) { m <- store$get_matrix(k); m[] <- round(runif(N11 * M11, 30, 100)); assign(k, m, envir = store$mats) }
})[["elapsed"]]
store_mb <- function(st) round(sum(sapply(st$component_names, function(k) as.numeric(object.size(st$get_matrix(k))))) / 1024^2, 1)
cat("Build time (s):", round(t_build, 2), "| matrices:", length(store$component_names), "each", paste(dim(store$get_matrix("CA1")), collapse = " x "), "\n")
cat("Store size (MB):", store_mb(store), "\n")

cat("\n--- Experiment 1: frequent individual-record updates ---\n")
arr11 <- array(unlist(mget(comp11, envir = store$mats), use.names = FALSE), dim = c(N11, M11, 5), dimnames = list(ids11, sub11, comp11))
upd_arr <- function(a, i, j, k, v) { a[i, j, k] <- v; a }
n_slow <- 10; n_fast <- 1000
pos <- cbind(sample(N11, n_fast), sample(M11, n_fast, TRUE), sample(5, n_fast, TRUE)); val <- sample(30:100, n_fast, TRUE)
t_copy <- system.time({ a <- arr11; for (u in seq_len(n_slow)) a <- upd_arr(a, pos[u, 1], pos[u, 2], pos[u, 3], val[u]) })[["elapsed"]] / n_slow
a2 <- arr11
t_inplace <- system.time(for (u in seq_len(n_fast)) a2[pos[u, 1], pos[u, 2], pos[u, 3]] <- val[u])[["elapsed"]] / n_fast
v1 <- StoreV1$new(marks = mget(comp11, envir = store$mats))
t_v1 <- system.time(for (u in 1:20) v1$update(comp11[pos[u, 3]], pos[u, 1], pos[u, 2], val[u]))[["elapsed"]] / 20
rm(v1); invisible(gc())
t_store <- system.time(for (u in seq_len(n_fast))
  store$update_mark(ids11[pos[u, 1]], sub11[pos[u, 2]], comp11[pos[u, 3]], val[u]))[["elapsed"]] / n_fast
upd <- data.frame(method = c("A copy 3-D array in a function", "B unchecked in-place edit",
                             "C RC v1 (list field, <<-)", "D RC v2 AssessmentStore (detach-edit)"),
                  ms_per_update = round(c(t_copy, t_inplace, t_v1, t_store) * 1000, 3),
                  validated = c("no", "no", "no", "yes"), audit_log = c("no", "no", "no", "yes"))
print(upd, right = FALSE)
cat("v1 -> v2 refinement: ", round(t_v1 / t_store), "x faster per update, with validation and audit log added.\n", sep = "")
```

```
cat("v2 costs ", round(t_store / t_inplace, 1), "x an unchecked in-place edit; that overhead buys validation + logging.\n", sep = "")
cat("Audit entries recorded:", length(store$audit), "| last entry:\n"); str(store$audit[[n_fast]])

cat("\n--- Experiment 2: adding a continuous-assessment component (CA4) ---\n")
new_mat <- matrix(round(runif(N11 * M11, 30, 100)), N11, M11, dimnames = list(ids11, sub11))
t_add_arr  <- system.time(arr_new <- array(c(arr11, new_mat), dim = c(N11, M11, 6), dimnames = list(ids11, sub11, c(comp11, "CA4"))))[["elapsed"]]
w_new <- c(CA1 = .08, CA2 = .08, CA3 = .08, CA4 = .06, model = .40, practical = .30)
t_add_store <- system.time(store$add_component("CA4", new_mat, w_new))[["elapsed"]]
cat("Add component -> rebuild array:", round(t_add_arr, 3), "s | store$add_component:", round(t_add_store, 3), "s\n")
rm(arr_new, a, a2); invisible(gc())

cat("\n--- Experiment 3: weighted totals for 100,000 x 40 ---\n")
t_tot <- system.time(tot11 <- store$weighted_total())[["elapsed"]]
cat("weighted_total():", round(t_tot, 3), "s | result dim:", paste(dim(tot11), collapse = " x "), "\n")
avg11 <- rowMeans(tot11); grade11 <- assign_grade(avg11)
cat("Grade distribution for 100,000 students:\n"); print(table(grade11))

cat("\n--- Integration with the existing framework ---\n")
one <- ids11[2500]
s4_one <- new("Student", reg_no = "192324006", name = "From store", programme = "IT", attendance = 82,
              marks = setNames(as.numeric(tot11[one, 1:5]), sub11[1:5]))
show(s4_one); str(eligibility(s4_one))
sample_ids <- sample(ids11, 50)
lg <- store$to_long(sample_ids)
bk <- xtabs(mark ~ student + subject + component, data = lg)
rt <- all(sapply(store$component_names, function(k) isTRUE(all.equal(unclass(bk)[sample_ids, sub11, k], store$get_matrix(k)[sample_ids, ], check.attributes = FALSE))))
cat("to_long() -> xtabs() round trip for 50 sampled students identical:", rt, "\n")

cat("\n--- Rejected updates (validation still enforced at scale) ---\n")
try_up <- function(label, ...) cat(sprintf("%-24s -> %s\n", label, tryCatch({ store$update_mark(...); "ACCEPTED" }, error = function(e) conditionMessage(e))))
try_up("mark = 150", "S000001", "SUB01", "CA1", 150)
```

```
try_up("unknown student", "S999999", "SUB01", "CA1", 50)
try_up("unknown component", "S000001", "SUB01", "CA9", 50)
try_up("text value", "S000001", "SUB01", "CA1", "AB")

cat("\n--- Acceptance criteria re-tested on the revised design ---\n")
c1 <- !safe_convert(c("85", "AB"), "numeric")$ok
c2 <- all(!sapply(list(function() store$update_mark("S000001", "SUB01", "CA1", 150), function() store$update_mark("NOPE", "SUB01", "CA1", 50)),
                  function(f) tryCatch({ f(); TRUE }, error = function(e) FALSE)))
c3 <- all(sapply(store$component_names, function(k) { m <- store$get_matrix(k); !is.null(rownames(m)) && !is.null(colnames(m)) }))
c5 <- t_tot < 10 && store_mb(store) < 1024
c6 <- identical(store$get_matrix("CA1")[1:5, 1:3], arr11[1:5, 1:3, "CA1"]) && "CA4" %in% store$component_names && abs(sum(store$weights) - 1) < 1e-8
acc <- data.frame(id = paste0("C", 1:6),
  criterion = c("Type safety", "Validity", "Interpretability", "Round-trip integrity", "Scalability", "Maintainability"),
  evidence = c("safe_convert() flags 'AB'", "bad value / unknown ID rejected", "every matrix has dimnames",
               "to_long() -> xtabs() identical", sprintf("weighted_total %.2f s, store %.0f MB", t_tot, store_mb(store)),
               "one-call update; CA4 added safely"),
  result = ifelse(c(c1, c2, c3, rt, c5, c6), "PASS", "FAIL"))
print(acc, right = FALSE)

cat("\n--- Cost of holding 100,000 S4 Student objects (extrapolated from Task 7) ---\n")
cat("Measured 2,000 S4 objects in", round(t_s4, 2), "s ->", round(t_s4 / 2000 * N11, 1), "s for 100,000 (linear extrapolation)\n")

png("fig/fig_t11_updates.png", width = 1600, height = 950, res = 170)
par(mar = c(5, 6, 4, 1))
barplot(upd$ms_per_update, log = "y", names.arg = c("A copy\n3-D array", "B unchecked\nin-place", "C RC v1\n(list field)", "D RC v2\n(detach-edit)"),
        col = c("#fc9272", "#9ecae1", "#fdd0a2", "#74c476"), ylab = "ms per update (log scale)", ylim = c(min(upd$ms_per_update) / 3, max(upd$ms_per_update) * 3),
        main = "Cost of one record update at 100,000 students x 40 subjects")
text(c(0.7, 1.9, 3.1, 4.3), upd$ms_per_update * 1.5, upd$ms_per_update)
dev.off()
```

Output (captured from running the code above)

```
--- Decision table: retain / modify / replace / redesign ---
  component                       action
1 Typed core vectors + validation RETAIN
2 safe_convert / strict_combine   RETAIN
3 S4 Student class                MODIFY (boundary use only)
4 3-D assessment array            REDESIGN (matrices in env)
5 Immutable update by copying     REPLACE (Reference Class)
6 Function library (Task 8)       MODIFY (per-component weights)

--- Build the store at the changed scale ---
Build time (s): 0.98 | matrices: 5 each 100000 x 40
Store size (MB): 183.1

--- Experiment 1: frequent individual-record updates ---
  method                              ms_per_update validated audit_log
1 A copy 3-D array in a function        160.700       no        no
2 B unchecked in-place edit               0.114       no        no
3 C RC v1 (list field, <<-)              14.500       no        no
4 D RC v2 AssessmentStore (detach-edit)   0.134       yes       yes
v1 -> v2 refinement: 108x faster per update, with validation and audit log added.
v2 costs 1.2x an unchecked in-place edit; that overhead buys validation + logging.
Audit entries recorded: 1000 | last entry:
List of 5
 $ id       : chr "S011496"
 $ subject  : chr "SUB06"
 $ component: chr "practical"
 $ old      : num 50
 $ new      : int 62

--- Experiment 2: adding a continuous-assessment component (CA4) ---
Add component -> rebuild array: 0.28 s | store$add_component: 0 s

--- Experiment 3: weighted totals for 100,000 x 40 ---
```

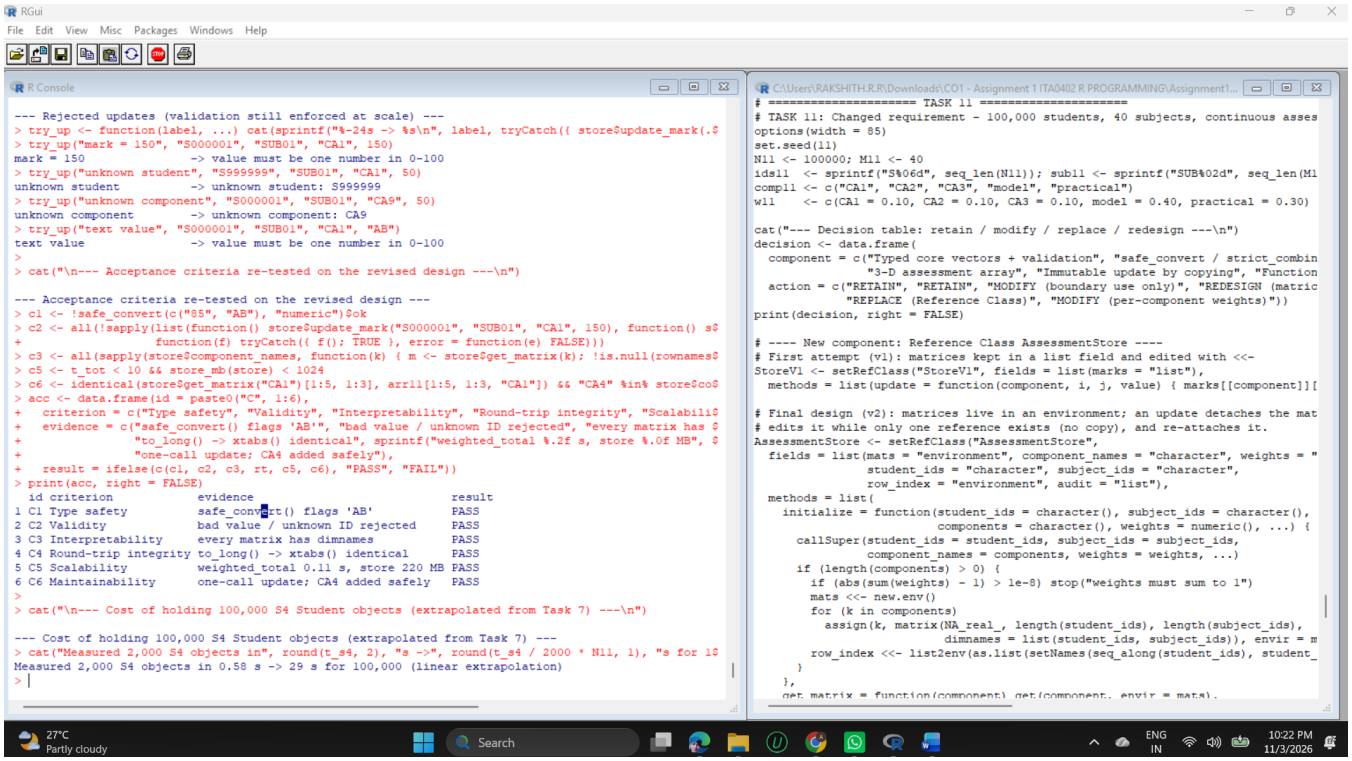
```
weighted_total(): 0.15 s | result dim: 100000 x 40
Grade distribution for 100,000 students:
grade11
   F     C     B     A     O
   0     0 99858   142     0

--- Integration with the existing framework ---
<Student> 192324006 - From store ( IT )
  attendance: 82 % | average mark: 59.46
  marks: SUB01=67.72, SUB02=47.42, SUB03=44.52, SUB04=47.98, SUB05=89.66
List of 2
 $ eligible: logi TRUE
 $ reasons : chr(0)
to_long() -> xtabs() round trip for 50 sampled students identical: TRUE

--- Rejected updates (validation still enforced at scale) ---
mark = 150              -> value must be one number in 0-100
unknown student         -> unknown student: S999999
unknown component       -> unknown component: CA9
text value              -> value must be one number in 0-100

--- Acceptance criteria re-tested on the revised design ---
  id criterion           evidence                          result
1 C1 Type safety         safe_convert() flags 'AB'         PASS
2 C2 Validity            bad value / unknown ID rejected   PASS
3 C3 Interpretability    every matrix has dimnames         PASS
4 C4 Round-trip integrity to_long() -> xtabs() identical   PASS
5 C5 Scalability         weighted_total 0.15 s, store 220 MB PASS
6 C6 Maintainability     one-call update; CA4 added safely  PASS

--- Cost of holding 100,000 S4 Student objects (extrapolated from Task 7) ---
Measured 2,000 S4 objects in 0.44 s -> 21.9 s for 100,000 (linear extrapolation)
```



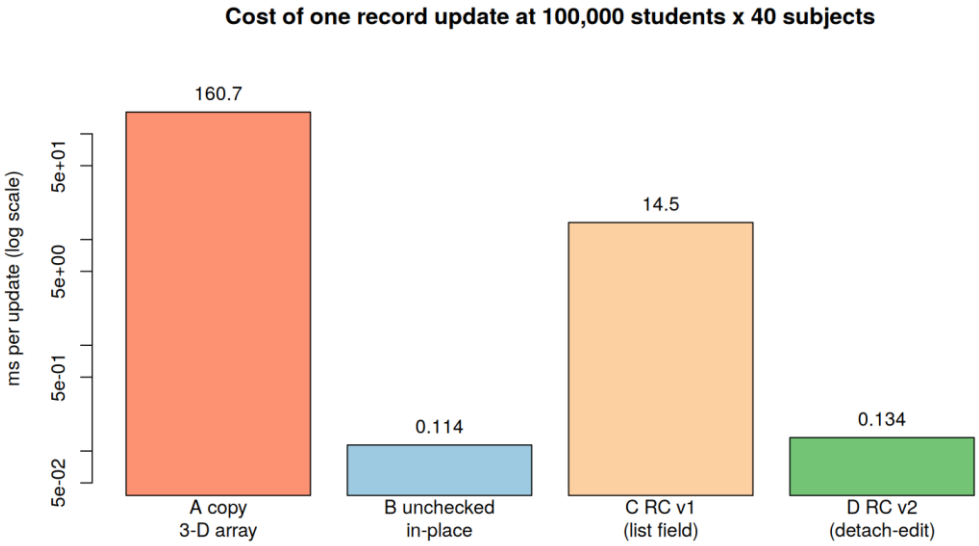


Figure 7: Cost of one record update at 100,000 students x 40 subjects (log scale)

11.3 Evidence-based defence

Dimension	Evidence	Source
Update performance	Copying the array: 160.7 ms per update. List-field store (v1): 14.5 ms. Final store: 0.134 ms, which is 108 times faster than v1 and about the	Experiment 1

	same as an unchecked in-place edit (0.114 ms), now with validation and an audit log.	
Extensibility	Adding CA4: rebuilding the array took 0.28 s, add_component() took essentially no time and left existing data untouched.	Experiment 2
Scalability	Weighted total for 100,000 x 40: 0.15 s. The store with six components needs 220 MB, well below the 1 GB criterion.	Experiment 3
Validation	Marks of 150, text values, an unknown student and an unknown component were all rejected.	Rejected updates
Semantic correctness	IDs stay character; marks stay numeric; to_long() followed by xtabs() reproduced the matrices for 50 sampled students.	Integration
Maintainability	One call to update, one call to add a component; the S4 Student and eligibility() still work on data taken from the store.	Integration
Acceptance criteria	C1 to C6 all PASS on the revised design.	Acceptance table

11.4 Trade-offs and limitations that remain

- **Memory:** all matrices live in RAM (183 MB for five components, 220 MB with six). Growth in students, subjects or components increases this linearly; a sparse or disk-based store would be needed well beyond this scale.
- **Reference semantics:** the store is modified in place, so two variables pointing to the same store see each other's changes. Copies must be made explicitly with $copy(), and the audit log is held in memory only, with no locking for several simultaneous users.
- **Reliance on reference counting:** the fast detach-edit update depends on how R manages copies. It was measured on R 4.3.3 and should be re-timed on the production R version.
- **Validation split:** full S4 validity is applied to single records and boundaries, while the store applies a lighter check on each update. This is faster but means the S4 class is not the only guard of consistency.

Future improvements: store the data in a database such as SQLite and load chunks on demand; use sparse matrices where many assessments are empty; write the audit log to a file; add an automated unit-test suite (for example with testthat) so that the test cases of Tasks 8 and 9 run on every change.

Conclusion. The retained parts (typed vectors, validators, named matrices, factors, data frames) keep their value under the new scale, and the redesigned parts (matrices per component, mutable validated store) address exactly the points where measurement showed a problem. The final architecture satisfies all six acceptance criteria with evidence.